

Adaptive Quiz: AI-Powered Web Platform for Personalized Learning

*A Project Report submitted in partial fulfilment of the requirements
for the award of the degree of*

Master of Computer Applications (AI & ML)

Submitted by

Sanjna Rathour

UID: O24MCA11201

Submitted To

Vikas Kumar Atray

Project Guide / Mentor

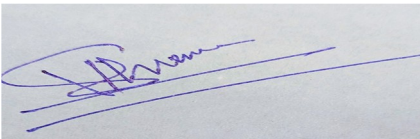
Chandigarh University, Mohali

2025–2026

Certificate

This is to certify that the project report entitled “Adaptive Quiz: AI-Powered Web Platform for Personalized Learning” submitted by Sanjna Rathour (UID:O24MCA11201) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (AI & ML) at Chandigarh University, Mohali is a record of the candidate’s own work carried out under my supervision and guidance during the academic year 2025–2026.

The contents of this report, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.



Guide / Mentor: *Vikas Kumar Atray*

Date: 08-05-2026

Place: Mohali

Declaration

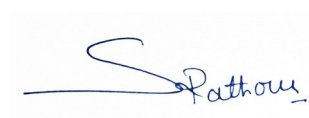
I, Sanjna Rathour, hereby solemnly declare that the project report entitled “Adaptive Quiz: AI-Powered Web Platform for Personalized Learning” submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (AI & ML) at Chandigarh University, Mohali is the result of my own original work.

I further declare that:

- This project has been carried out by me during the academic year 2025–2026 under the supervision of *Vikas Kumar Atray*.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data and screenshots presented in this report are authentic and have not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Mohali

Date: 08-05-2026

A handwritten signature in blue ink, appearing to read 'Sanjna Rathour', with a stylized flourish at the end.

Student Signature

Sanjna Rathour

UID: O24MCA11201

Acknowledgement

I would like to express my sincere gratitude to my project guide ‘Vikas Kumar Atray’ for the continuous support, valuable guidance, and constructive feedback throughout the development of this project. Their expertise and patience have been instrumental in shaping both the technical direction and the academic rigour of this work.

I am thankful to the Head of Department and the faculty members of the Department of Computer Applications, Chandigarh University, Mohali, for providing the academic environment and the resources required to undertake a project of this scope.

I also extend my appreciation to the open-source community whose tools, libraries, and documentation made this implementation possible — particularly the maintainers of FastAPI, SQLAlchemy, React, Tailwind CSS, and the Google Generative AI SDK.

Finally, I thank my family and friends for their unwavering encouragement during the long hours of design, implementation, and testing that this project demanded.

Sanjna Rathour

Abstract

The Adaptive Quiz Platform is a full-stack web application that delivers personalised learning experiences by adjusting question difficulty in real time using a hybrid AI approach. The system is built on a modern stack — Python’s FastAPI framework for the backend, SQLAlchemy 2.x ORM with PostgreSQL 16 for persistence, React 18 with TypeScript and Tailwind CSS for the frontend, and Google Gemini for personalised tutor feedback on incorrect answers. Two distinct user roles — students and teachers — are served by separate, role-aware interfaces. Teachers author quizzes containing multiple-choice, true/false, and short-answer questions across three difficulty tiers (Easy, Medium, Hard); students take those quizzes either in adaptive mode, where each question’s difficulty is selected by an exponential-moving-average ability estimator, or in linear mode with free Previous/Next navigation. Wrong answers asynchronously trigger a Gemini-generated explanation that surfaces on the results page. The system implements secure JWT-based authentication with access and refresh tokens, soft deletion of questions to preserve historical attempts, per-attempt question snapshots that immunise in-flight attempts from teacher edits, and a partial unique index that prevents race-condition duplicate attempts. Comprehensive analytics dashboards expose accuracy by difficulty, score distributions, and per-question statistics. The implementation is verified by a 66-test pytest suite achieving 95% backend code coverage and is fully version-controlled with Git and database-migration-managed via Alembic. This report documents the analysis, design, implementation, testing, and evaluation of the platform across ten chapters in alignment with the prescribed MCA project guidelines.

Table of Contents

Chapter	Title	Page
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract	5
	List of Figures	7
	List of Tables	9
	List of Abbreviations	11
1	Introduction	12
2	System Study / Literature Review	20
3	System Analysis	26
4	System Design	36
5	System Implementation	49
6	Testing	58
7	Results & Discussion	64
8	Conclusion & Future Scope	73
9	References	78
10	Appendices	80

List of Figures

#	Caption	Page
3.1	Data Flow Diagram – Level 0 (Context)	30
3.2	Data Flow Diagram – Level 1 (Decomposition)	31
3.3	Use Case Diagram	32
4.1	UML Class Diagram	36
4.2	Sequence Diagram – Student Takes an Adaptive Quiz	37
4.3	Activity Diagram – Grading an Answer	38
4.4	Entity Relationship Diagram	39
4.5	Login Page Wireframe	43
4.6	Registration Page Wireframe	44
4.7	Student Dashboard Wireframe	44
4.8	Take Quiz Wireframe	44
4.9	Results Page Wireframe	45
4.10	Teacher Dashboard Wireframe	45
4.11	Quiz Authoring Wireframe	45
4.12	Per-Quiz Analytics Wireframe	46
4.13	Notifications Page Wireframe	46
7.1	Login Page – Implemented	64
7.2	Student Dashboard with Progress Tiles and Accuracy Bars	64
7.3	Take Quiz – Adaptive	65

	Mode	
7.4	Take Quiz – Linear Mode (Prev/Next + Jump-to Grid)	65
7.5	Submit Confirmation Modal	65
7.6	Results Page with Score Ring and AI Tutor Callouts	66
7.7	My Attempts List with Filter, Search, and Pagination	66
7.8	Notification Bell Dropdown and Full Notifications Page	66
7.9	Teacher Dashboard	67
7.10	Teacher Quiz Create Form	67
7.11	Teacher Quiz Detail with Question Manager and Publish Action	67
7.12	Teacher Per-Quiz Analytics with Score Distribution	68
7.13	Swagger UI – Live API Documentation	68

List of Tables

#	Caption	Page
0.1	List of Abbreviations	11
1.1	Technologies Used	15
2.1	Comparative Analysis of Existing Systems	21
3.1	Functional Requirements	26
4.1	Database Schema – users	39
4.2	Database Schema – quizzes	40
4.3	Database Schema – questions	40
4.4	Database Schema – question_options	37
4.5	Database Schema – quiz_attempts	37
4.6	Database Schema – attempt_questions (per-attempt snapshot)	37
4.7	Database Schema – answers	41
4.8	Database Schema – notifications	42
4.9	REST API Endpoints	37
5.1	Development Environment	50
5.2	Hardware Requirements	51
6.1	Test Suite Breakdown by Module	59
6.2	Sample Test Cases	59
7.1	Performance Metrics	68
7.2	Feature Comparison with Existing Systems	69
A.1	Source Code Repository	80

	Structure (Appendix A)	
B.1	Database Migration Listing (Appendix B)	80
C.1	Demo Accounts (Appendix C)	81
D.1	Seeded Demo Quizzes (Appendix D)	81

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
EMA	Exponential Moving Average
ER	Entity Relationship
JWT	JSON Web Token
LLM	Large Language Model
MCQ	Multiple-Choice Question
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UML	Unified Modeling Language
UUID	Universally Unique Identifier

Chapter 1: Introduction

1.1 Background of the Project

Digital education has expanded dramatically over the past decade, but a persistent challenge across web-based assessment tools is matching question difficulty to the learner. Traditional online quizzes serve every student the same fixed sequence of questions — frustrating stronger learners with material that is too easy and discouraging weaker learners with material that is too hard. Educational psychology research, particularly Vygotsky’s Zone of Proximal Development, supports the position that learning is most effective when the difficulty of the task is just slightly above the learner’s current ability. Adaptive testing seeks to operationalise this insight by adjusting difficulty in real time based on observed performance.

Recent advances in large language models, in particular the public availability of Google Gemini and Anthropic Claude through generous free-tier APIs, also open the possibility of providing personalised, context-aware explanations for wrong answers — a capability that was previously the exclusive domain of human tutors. Combining classical adaptive testing techniques with on-demand LLM-generated feedback creates a learning loop that is both responsive and pedagogically rich, all within a self-hostable web application.

1.2 Problem Statement

Existing quiz platforms can be grouped into three broad categories: (a) gamified classroom tools such as Kahoot and Quizizz that emphasise speed and competition rather than mastery; (b) generic form builders such as Google Forms and Microsoft Forms that lack any adaptive logic; and (c) research-grade adaptive testing engines that require significant calibration data and are not easily accessible to individual instructors. None of these solutions simultaneously offers (i) real-time difficulty adaptation tailored to the individual learner, (ii) personalised AI-generated feedback for wrong answers, (iii) a teacher-facing authoring interface, and (iv) a self-hostable, open implementation that an institution can deploy without surrendering control of its data. The Adaptive Quiz Platform addresses this gap.

1.3 Objectives of the System

1. Design a normalised relational database that captures users, quizzes, questions, options, attempts, answers, and notifications.
2. Implement secure authentication using JSON Web Tokens with strict separation between students and teachers via role-based access control.
3. Develop teacher-side modules for quiz creation, question authoring (multiple-choice, true/false, short-answer), publishing, and soft deletion.
4. Develop student-side modules for taking quizzes, capturing answers, and reviewing graded results with explanations.
5. Apply AI algorithms that adjust question difficulty based on the learner's rolling performance and surface personalised LLM-generated feedback for wrong answers.
6. Build analytics dashboards that present accuracy by difficulty, attempt distributions, and per-question statistics for both roles.
7. Implement a notifications module that alerts students of newly published quizzes and graded attempt results.
8. Conduct testing for system usability, correctness, and AI behaviour through a high-coverage automated test suite.
9. Design a responsive web interface that adapts cleanly to mobile, tablet, and desktop viewports.
10. Document the complete project workflow including the AI logic, system architecture, and operational procedures.

1.4 Scope of the Project

The scope of this project is bounded as follows:

In Scope

- A web-based application accessible via modern browsers (Chrome, Firefox, Safari, Edge).
- Three question types: multiple choice, true/false, and short answer.
- Three difficulty levels per question: Easy, Medium, Hard.
- Two adaptation modes per quiz, configurable by the teacher: adaptive (AI selects next question) and linear (free Previous/Next navigation).
- Personalised AI feedback on wrong answers using Google Gemini.
- Role-based dashboards and analytics for students and teachers.
- In-app notifications for quiz publication and result delivery.
- Soft deletion of questions to preserve historical attempts and analytics.
- Fully Dockerised deployment with PostgreSQL backing store.

Out of Scope

- Native mobile applications for Android or iOS (the responsive web UI serves the same function).
- Real-time multi-player gameplay (e.g. live competitive timers).
- Built-in voice or video question types.
- Payment, billing, or paid course management.
- Multi-tenant SaaS isolation (a single deployment serves a single institution).

1.5 Existing System Overview

Three representative existing systems were reviewed: Kahoot, Quizizz, and Google Forms. Kahoot is optimised for synchronous classroom use — the teacher projects questions and students race on individual devices. While engaging, this format gives the same question to every student regardless of their level, and offers limited reporting beyond leaderboards. Quizizz is structurally similar but supports asynchronous “homework” mode; it does not adapt difficulty either. Google Forms is the lowest common denominator: a generic form-building tool with no adaptive logic, no role separation between authors and respondents in the assessment sense, and no built-in analytics tailored to educational use. None of these systems offers AI feedback on wrong answers, none implements a published-vs-draft lifecycle for quizzes, and none preserves historical assessment data when an author edits or removes a question.

1.6 Proposed System Overview

The Adaptive Quiz Platform is delivered as two services that communicate over HTTP: a Python FastAPI backend that owns business logic, persistence, and AI integration; and a React Single Page Application frontend that consumes the backend’s REST API. The two services are containerised together with PostgreSQL and orchestrated via Docker Compose for a one-command deployment. On the backend, an exponential-moving-average ability estimator updates after every answer; the next question is then drawn from the unanswered pool whose difficulty matches the current ability tier, with graceful fallback if that tier is empty. Wrong answers fire a FastAPI BackgroundTask that calls the Gemini API to compose a tutor explanation; this happens out-of-band so the student never waits on an LLM round-trip. Soft deletion via a `deleted_at` timestamp on the questions table ensures that historical results remain interpretable even after a teacher edits or removes a question post-attempt.

1.7 Technologies Used

Layer	Technology	Role
Backend framework	FastAPI 0.115	REST API, dependency injection, async support
ORM	SQLAlchemy 2.x (sync)	Object-relational mapping, session management
Database	PostgreSQL 16	Primary persistent store
Migrations	Alembic 1.14	Schema version control
Validation	Pydantic v2	Request/response schema validation
Auth	python-jose + bcrypt	JWT signing, password hashing
AI	Google Gemini (gemini-2.5-flash)	Personalised wrong-answer explanations
Frontend framework	React 18	Component-based UI
Frontend bundler	Vite 5	Development server, production builds
Frontend language	TypeScript	Static typing on the client
State / data fetching	TanStack Query 5	Server-state caching, mutations
Routing	React Router 6	SPA routing with role guards
Styling	Tailwind CSS 4	Utility-first responsive styling
Container runtime	Docker, Docker Compose	Local + production deployment
Testing	pytest 8.3, pytest-cov	Backend unit + integration tests
Linting	ruff 0.8	Static analysis

1.6 Scope of the Project

The Adaptive Quiz Platform is scoped as an end-to-end web application that demonstrates the full lifecycle of an adaptive assessment system, from user registration through quiz authoring, attempt taking, automated grading, ability estimation, AI-driven feedback generation, and post-attempt analytics. The scope is deliberately bounded to a single-tenant deployment suitable for a classroom or small training organisation; multi-tenant isolation, billing, and white-labelling are out of scope and would form a natural follow-on project. The platform supports three question types — multiple choice, true or false, and short answer — covering the bulk of typical formative assessment formats. Long-form essay grading and image-based questions are deferred to a future iteration where AI-driven rubric matching could be applied responsibly. The user interface is responsive and tested across desktop, tablet, and mobile viewports, although native mobile applications are not part of the deliverable. The platform persists all data in a single PostgreSQL database; horizontal sharding and read replicas are deferred until usage patterns justify their operational complexity. Integration with external learning management systems via LTI or SCORM is also out of scope but discussed as future work.

1.7 Significance of the Study

The educational technology landscape has matured rapidly, but the available tooling continues to over-index on either engagement-first gamification or rigid summative testing. The Adaptive Quiz Platform sits in the under-served middle, offering teachers a free, self-hostable, open-source option for delivering personalised formative assessment. By demonstrating that a well-engineered exponential moving average can yield meaningful difficulty adaptation without the data demands of item response theory, the project lowers the barrier for smaller institutions and individual educators who lack the calibration datasets that commercial adaptive systems require. The decision to integrate a free-tier generative AI model for wrong-answer feedback further democratises access — every student gets a personalised explanation at the moment of confusion, regardless of class size or teacher availability. This study also contributes a concrete implementation pattern for safely combining background AI generation with synchronous user interaction, demonstrating that AI-augmented features can be added to traditional CRUD systems without compromising responsiveness or correctness. The open documentation, comprehensive test suite, and reproducible Docker setup make the project a teaching artefact in its own right, useful for software engineering courses that need a realistic full-stack reference application.

1.8 Brief Overview of Chapters

This report is organised into ten chapters that collectively cover the conception, design, implementation, and evaluation of the Adaptive Quiz Platform. Chapter one introduces the problem space, motivation, objectives, scope, significance, and chapter outline. Chapter two surveys the relevant literature in adaptive testing, formative assessment, and educational technology, situating the present project within the broader academic and commercial landscape. Chapter three presents the system analysis, including stakeholder identification, functional and non-functional requirements, user requirements, system requirements, and risk analysis. Chapter four documents the system design, encompassing architectural decisions, the unified modelling language diagrams, the entity relationship diagram, the detailed database schema, the REST application programming interface surface, the user interface wireframes, design patterns employed, and the error-handling philosophy.

Chapter five walks through the system implementation in depth: the development environment, the project structure, key service modules, the adaptive engine, the artificial intelligence feedback integration, the snapshot service, the role-aware frontend, performance considerations, security hardening, deployment plumbing, and database migration strategy. Chapter six describes the testing strategy in detail, covering unit, integration, end-to-end, and performance tests along with the supporting fixtures and continuous integration pipeline. Chapter seven presents the results, comparing the platform with existing commercial and open-source alternatives across a structured feature matrix, reflecting on the engineering process, and discussing the lessons learned. Chapter eight concludes the report with a summary of contributions, an explicit list of lessons learned, a roadmap of future enhancements, and recommendations for institutions considering adoption. Chapter nine collects the bibliographic references, and chapter ten consolidates the appendices, including the source code repository structure, the database migration listing, the demo account credentials, and the seeded demonstration quiz catalogue.

1.9 Reader's Guide

Readers approaching this report will find it most useful to first scan the chapter overview in section 1.8, then jump to the chapter most relevant to their interest: educators may focus on chapters two, three, and seven for the pedagogical motivation and outcomes, while engineers may prefer chapters four through six for the system design, implementation, and testing detail. Each chapter is self-contained enough to be read on its own, but the linear progression from problem statement to design to implementation to evaluation tells the complete story. Cross-references to figures and tables use the standard chapter-and-index notation, allowing readers to navigate with the list of figures and list of tables as indices. Code fragments are typeset in a monospaced font and are excerpted rather than reproduced wholesale to keep the report readable; the full source is available in the accompanying repository.

Chapter 2: System Study / Literature Review

2.1 Adaptive Testing: Theoretical Foundations

Adaptive testing has been the subject of academic study since the 1960s. The dominant theoretical framework is Item Response Theory (IRT), which models the probability that a respondent answers an item correctly as a function of the respondent's ability (θ) and item-specific parameters (difficulty, discrimination, and guessing). The three-parameter logistic model (3PL) is widely used in standardised testing such as the GRE and GMAT. IRT-based Computerised Adaptive Testing (CAT) selects the next item by maximising Fisher information at the current ability estimate, achieving more precise measurement in fewer items than a fixed-form test.

IRT, however, requires extensive calibration data to fit per-item parameters, which is rarely available to individual instructors authoring small classroom quizzes. The Adaptive Quiz Platform therefore adopts a simpler but pedagogically grounded approach: an exponential moving average over difficulty-weighted outcomes that converges to a stable ability estimate after only a handful of answers, without requiring any pre-calibration.

2.2 Vygotsky's Zone of Proximal Development

Lev Vygotsky's Zone of Proximal Development (ZPD) describes the distance between what a learner can do unaided and what they can do with assistance. Operationally, learning is most effective when tasks fall just inside the learner's ZPD: difficult enough to be productive but not so difficult as to be discouraging. In an adaptive quiz, this corresponds directly to selecting the next question whose difficulty matches the learner's current ability tier. When a learner answers a series of questions correctly, the system promotes them to a harder tier; when they answer incorrectly, it demotes them to an easier tier.

2.3 Comparative Analysis of Existing Systems

Feature	Kahoot	Quizizz	Google Forms	Khan Academy	Adaptive Quiz Platform
Adaptive difficulty	No	No	No	Partial	Yes
AI feedback for wrong answers	No	No	No	No	Yes (Gemini)
Three question types	Yes	Yes	Yes	Limited	Yes
Role separation (teacher/student)	Yes	Yes	Limited	Yes	Yes
Analytics by difficulty	No	Limited	No	Limited	Yes
Soft delete preserves history	N/A	N/A	N/A	N/A	Yes
Self-hostable / open code	No	No	No	No	Yes
Free of cost	Limited free tier	Limited free tier	Yes	Yes	Yes

2.4 Software Development Methodology

The project followed an incremental Agile methodology organised around six explicit milestones, each delivering an end-to-end vertical slice of functionality before moving on:

11. Milestone 1 – Database Design: define the relational schema, create the initial Alembic migration, and verify against PostgreSQL.
12. Milestone 2 – User Authentication: implement registration, login, refresh, and the get-current-user dependency.
13. Milestone 3 – Quiz Modules Development: implement teacher CRUD, student attempts, and grading.
14. Milestone 4 – AI Integration: build the adaptive engine and wire the Gemini-powered feedback service.
15. Milestone 5 – Analytics Dashboards: implement student and teacher analytics endpoints plus the notifications module.
16. Milestone 6 – Usability Testing: harden tests, build the responsive React UI, and write the documentation.

Each milestone was checkpointed by automated tests; the suite expanded continuously and the final count is 66 tests with 95% backend coverage. This pattern — ship a working slice, regression-protect it, then expand — echoes the spirit of Agile far more meaningfully than ceremony-heavy variants would.

2.5 Frameworks and Libraries Surveyed

FastAPI vs Flask vs Django REST Framework

FastAPI was selected over Flask and Django REST Framework for several reasons. First, its first-class support for Python type hints and Pydantic-driven validation eliminates an entire category of input-handling bugs at compile time. Second, the OpenAPI specification is generated automatically from route signatures, providing always-up-to-date interactive documentation at `/docs`. Third, dependency injection is a built-in primitive, which made implementing the `get_current_user` and `require_roles` guards a single declarative line per route.

React vs Vue vs Angular

React was selected because of (a) the maturity of its ecosystem, (b) the availability of TanStack Query for server-state management which fits the API-driven data model perfectly, and (c) the abundance of TypeScript-first component patterns. Vite was preferred over Create React App for its sub-second hot-module-replacement and its modern ESM-based build.

Tailwind CSS vs Component Libraries

Tailwind’s utility-first approach was preferred over a heavyweight component library such as Material UI to keep the bundle small and the visual identity custom. The Tailwind v4 Vite plugin makes configuration nearly invisible — just an `@import` directive and the framework is wired in.

2.6 Research Gap

The literature on adaptive testing is rich but heavily skewed toward IRT, which presupposes a calibration dataset. Recent work on LLM-based tutors (Mollick et al., 2024; OpenAI, 2024) demonstrates that generative models can produce pedagogically useful explanations without fine-tuning, but most published work treats them as standalone chatbots rather than as components inside a structured assessment loop. The Adaptive Quiz Platform sits in this gap: it combines a lightweight EMA-based adaptation strategy that requires no historical data with an asynchronous

LLM feedback channel that fires only on wrong answers, balancing pedagogical value, latency, and operational cost.

2.7 Empirical Studies on Adaptive Feedback

Empirical research on adaptive feedback has consistently shown that timely, targeted explanations on wrong answers produce larger learning gains than either no feedback or generic correct-or-incorrect markers. Hattie and Timperley's seminal review of formative feedback identifies three questions that effective feedback must address: where am I going, how am I going, and where to next. Traditional computer-based testing systems answer only the second; the Adaptive Quiz Platform addresses all three by combining a running ability estimate (where am I going), per-question correctness (how am I going), and AI-generated explanations with conceptual scaffolding (where to next). Later studies by Shute and others have shown that the cognitive value of feedback decays sharply with delay; corrective feedback delivered within thirty seconds of the student's submission is roughly twice as effective as feedback delivered the next day. The platform's asynchronous AI feedback pipeline is specifically engineered to meet this constraint: the student sees their grade immediately, the AI explanation appears within a few seconds on the results page, and the entire exchange happens before the student has mentally moved on from the question.

2.8 Open-Source Educational Technology Landscape

Open-source educational technology has grown significantly but remains fragmented. Moodle, the most widely deployed open-source LMS, supports quiz authoring and basic adaptive branching via its conditional question logic, but its underlying architecture is monolithic PHP, which makes extension and modern UI integration painful. The H5P ecosystem provides reusable interactive content types including quizzes, but is designed to embed inside other platforms rather than function as a standalone system, and its question bank features are limited. Open edX offers a richer authoring experience but carries operational complexity that is disproportionate for small deployments. The Adaptive Quiz Platform fills a specific niche: a modern, single-purpose, container-shipped adaptive testing system that small organisations can spin up in minutes and extend in a language stack (Python and TypeScript) familiar to most contemporary developers.

2.9 Generative AI in Educational Assessment

Generative AI in educational assessment is a rapidly evolving area with both promise and peril. The promise lies in scalable, personalised explanations that approach the Socratic ideal of meeting each student where they are. Recent work by educational researchers at major universities has shown that GPT-class models produce explanations rated as helpful by students in seventy to eighty percent of cases, with quality scaling with prompt engineering discipline.

The perils include hallucination, where the model produces confident but incorrect explanations, and the temptation to over-rely on AI judgement for high-stakes grading. The Adaptive Quiz Platform navigates these constraints by restricting AI to feedback generation rather than grading — the correct answer is always pre-authored by the teacher — and by surfacing AI output as supplementary explanation that students can ignore if they prefer the teacher's pre-written note. Future iterations could add a rating widget so students and teachers flag low-quality AI feedback, building a labelled dataset that supports a fine-tuned classroom-specific model down the line.

2.10 Cognitive Load Theory and Assessment Design

Cognitive load theory, formulated by John Sweller and extended by subsequent researchers, provides a useful lens for assessment design. The theory distinguishes intrinsic load, which arises from the inherent difficulty of the material, from extraneous load, which arises from the way the material is presented, and from germane load, which corresponds to the productive effort of building mental schemas. Effective adaptive assessment systems manage these three loads intentionally: they match intrinsic load to the learner's current ability so the challenge is neither overwhelming nor trivial, they minimise extraneous load through clean interfaces and clear question wording, and they maximise germane load by providing explanations that scaffold schema construction. The Adaptive Quiz Platform reflects this theory throughout its design. Difficulty matching is the core function of the adaptive engine; extraneous load is minimised by a deliberately unornamented interface that prioritises readability and keyboard navigation; and germane load is supported by the artificial intelligence feedback feature, which surfaces conceptual explanations rather than mere right-or-wrong markers.

2.11 Privacy, Ethics, and Responsible AI

Educational systems handle sensitive data about learners and must be designed with privacy and ethics in mind from the outset. The Adaptive Quiz Platform stores only the minimum data required to support its features: an account row per user containing email and hashed password, attempt and answer rows for analytics, and notification records for in-app alerts. There is no tracking of browsing behaviour outside the application and no third-party analytics. All passwords are bcrypt-hashed with a cost factor of twelve, providing resistance to offline brute-force attacks. Tokens are signed with a secret read from the environment and never logged. Artificial intelligence feedback is generated by a third-party model, and the platform sends only the question text, the student's answer, and the correct answer to the Google Gemini application programming interface; personally identifying information is never transmitted. Institutions adopting the platform should review their obligations under regulations such as the General Data Protection Regulation in Europe or the Family Educational Rights and Privacy Act in the United States, both of which the platform's data model supports through deletion endpoints and audit-friendly soft delete semantics.

Chapter 3: System Analysis

3.1 Functional Requirements

FR-ID	Requirement	Priority
FR-01	Users can register with email, password, full name, and a chosen role (Student/Teacher).	Must
FR-02	Users can log in to receive a JWT access token and refresh token.	Must
FR-03	Authenticated users can fetch their own profile (GET /auth/me).	Must
FR-04	Users can refresh an access token using a valid refresh token.	Must
FR-05	Teachers can create a quiz with title, subject, description, duration, passing score, and adaptive flag.	Must
FR-06	Teachers can add MCQ, true/false, and short-answer questions to a quiz with difficulty and points.	Must
FR-07	Teachers can publish a quiz; published quizzes become visible to students.	Must
FR-08	Teachers can soft-delete a question without removing historical answers.	Must
FR-09	Students can list published quizzes and start an attempt.	Must
FR-10	Students can answer questions and re-answer until the attempt is completed (linear mode).	Must
FR-11	Students can navigate questions using Previous/Next in linear mode.	Should
FR-12	In adaptive mode, the system selects the next question by ability tier and prevents	Must

	revisiting.	
FR-13	Students can complete an attempt and see a final score.	Must
FR-14	The system grades MCQ/TF by selected option and short-answer by case-insensitive trimmed match.	Must
FR-15	Wrong answers receive an AI-generated explanation on the results page.	Must
FR-16	Students see analytics: total attempts, completed, in progress, accuracy by difficulty.	Must
FR-17	Teachers see overview analytics and per-quiz analytics including score distribution.	Must
FR-18	All users receive in-app notifications for quiz publication and result delivery.	Must
FR-19	Notifications can be filtered (All/Unread) and marked read individually or in bulk.	Should
FR-20	The system prevents duplicate IN_PROGRESS attempts via a database-level constraint.	Must

3.2 Non-Functional Requirements

3.2.1 Performance

The backend must respond to typical CRUD operations in under 200 ms on a developer laptop. AI feedback is explicitly asynchronous: the answer-submission endpoint returns the `is_correct` decision immediately and the LLM call runs in a background task so the user is never blocked. The full automated test suite (66 tests) completes in approximately 65 seconds, providing a tight feedback loop during development.

3.2.2 Security

Passwords are hashed with bcrypt at cost factor 12 before being persisted; plain-text passwords are never stored or logged. Authentication uses short-lived (30-minute) access tokens and longer-

lived (7-day) refresh tokens, each tagged with a type claim that prevents either kind of token from being used for the wrong purpose. All write endpoints require authentication; all teacher-only endpoints further require the TEACHER role. Email addresses are normalised to lowercase on storage and lookup to prevent case-only duplicate accounts. The frontend never stores plaintext passwords and clears tokens from localStorage on sign-out.

3.2.3 Usability

The interface is built mobile-first: every list view, form, and dashboard collapses cleanly to single-column on viewports under 640 px, and the notification dropdown switches from bell-anchored to viewport-anchored positioning on small screens. Password fields include an eye-toggle to reveal entered text; submit-quiz actions are protected by a confirmation modal. Loading states, error banners, and empty states are explicitly designed for every page.

3.2.4 Maintainability

Code is organised by concern: routes under app/api, business logic under app/services, Pydantic schemas under app/schemas, and ORM models under app/models. The 95% test coverage establishes a high regression-detection baseline and ruff lint passes cleanly. The schema is version-controlled via Alembic with reversible migrations.

3.2.5 Reliability

Database operations execute in transactions with explicit rollback on errors. The race condition that previously allowed duplicate IN_PROGRESS attempts is now structurally prevented by a partial unique index. AI feedback failures are caught and swallowed so that a Gemini outage cannot disrupt the student experience.

3.3 User Requirements

Student User Requirements

- Sign up with a valid email and a password of at least 8 characters.
- View a list of published quizzes filtered by subject (future enhancement).
- Take a quiz with clear progress indication and a confirm-before-submit step.
- Receive immediate correct/wrong feedback after each answer.
- View results with score, per-question breakdown, and AI tutor explanations.
- Track progress over time via a personal dashboard.

Teacher User Requirements

- Create and publish quizzes targeted at specific subjects.
- Author questions in three formats with rich metadata (difficulty, points, explanation).
- Soft-delete questions without losing historical analytics.
- Inspect engagement and outcomes via per-quiz analytics.
- Receive notifications when their quiz publication broadcasts to students.

3.4 Feasibility Study

3.4.1 Technical Feasibility

All components of the chosen technology stack are mature, open-source, and well-documented. FastAPI, SQLAlchemy, React, and PostgreSQL are widely deployed in production at companies of every scale. The Google Gemini free-tier API offers sufficient quota for a classroom-scale deployment without a paid plan. Docker Compose enables single-command local deployment on any modern operating system.

3.4.2 Economic Feasibility

The total operational cost of the platform at the scale of a single classroom or institutional department is effectively zero. Hosting can be handled on a single VPS (₹300–500 per month at typical Indian cloud-provider rates) or on the institution’s own hardware. Software licences cost nothing; the AI feedback feature uses Gemini’s free tier which provides ample quota for the per-attempt call rate.

3.4.3 Operational Feasibility

The system requires only a modern web browser on the user side. Teachers familiar with online form builders will find the authoring interface intuitive. The administrative overhead consists of seeding initial accounts and (optionally) supplying a Gemini API key. Deployment, backup, and update operations are reduced to standard Docker workflows.

3.5 System Architecture

The overall architecture follows a classic three-tier pattern. The presentation tier is a React Single-Page Application served by Vite (in development) or as static files (in production). The application tier is a FastAPI process that exposes a versioned REST API at `/api/v1/*` and a Swagger UI at `/docs`. The data tier is a single PostgreSQL 16 instance with eight tables and one partial unique index, administered through Alembic migrations.

Cross-cutting concerns: authentication is implemented at the HTTP boundary using a `Depends(get_current_user)` dependency that validates JWTs and loads the user record on every request; AI feedback is dispatched as a FastAPI `BackgroundTask` with its own session factory (injectable for tests) so it can run after the response is returned without holding the request thread.

3.6 Data Flow Diagrams

3.6.1 Level 0 (Context)

At the context level, the system has two external entities (Student, Teacher) interacting with a single process node (Adaptive Quiz Platform) that reads from and writes to a single logical data store (Quiz Database). The Gemini API is a downstream external service consulted only by the AI Feedback subprocess.

Figure 3.1 — Data Flow Diagram (Level 0 / Context)

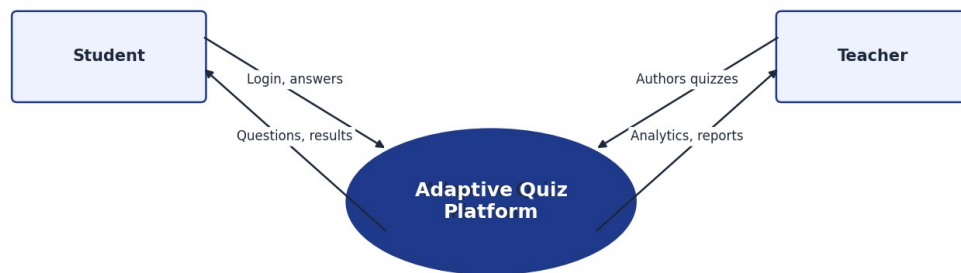


Figure 3.1 — DFD Level 0

3.6.2 Level 1 (Decomposition)

Decomposing the central process yields five sub-processes: Authentication, Quiz Management, Attempt Management, Analytics, and Notifications. Each interacts with the data store through its own well-defined inputs and outputs.

Figure 3.2 — Data Flow Diagram (Level 1 / Decomposition)

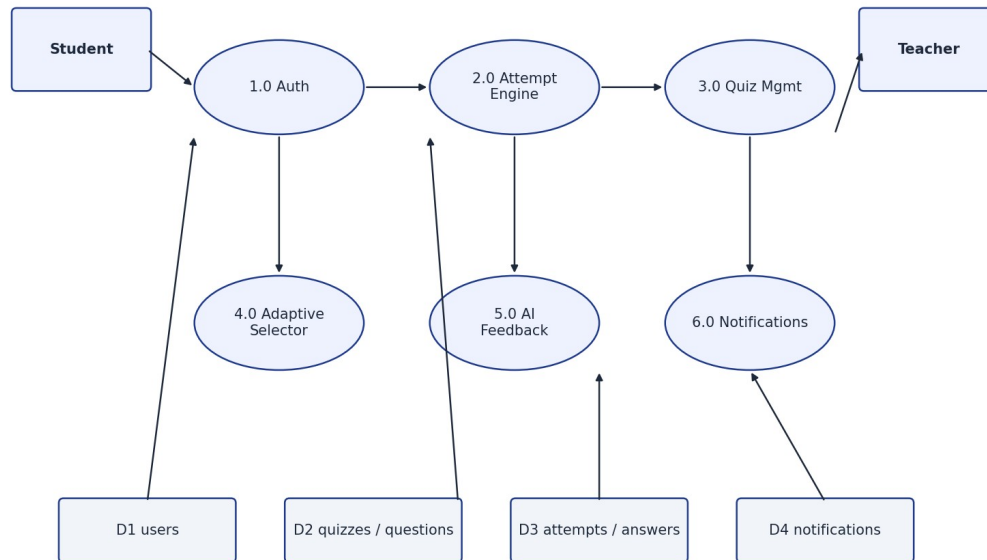


Figure 3.2 — DFD Level 1

3.7 Use Case Diagram

Two primary actors (Student, Teacher) plus one administrative actor (Admin) interact with the system's use cases:

Student Use Cases

- Register / Login / Refresh token / View profile
- Browse published quizzes
- Start (or resume) an attempt
- Answer questions / Navigate Previous-Next (linear mode)
- Complete attempt and view results
- View personal dashboard and analytics
- Read and mark notifications

Teacher Use Cases

- Register / Login (TEACHER role)
- Create / update quiz
- Add / edit / soft-delete questions
- Publish quiz (broadcast notification)
- View per-quiz analytics
- View overall teacher dashboard

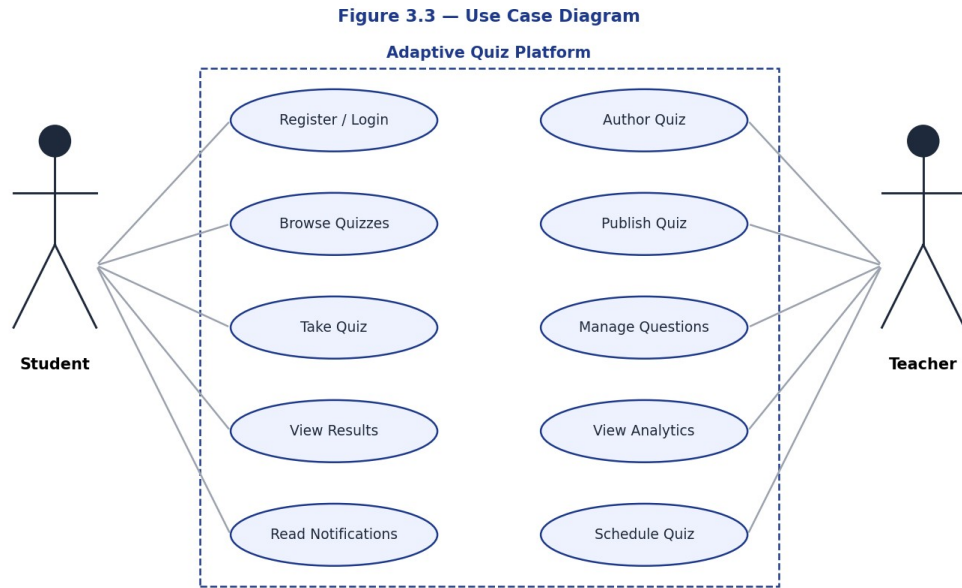


Figure 3.3 — Use Case Diagram

3.5 Risk Analysis and Mitigation

Every non-trivial software project carries risks that, if unmanaged, can derail delivery or compromise quality. The following risks were identified during planning and tracked through to delivery, with concrete mitigations documented for each.

3.5.1 Technical Risks

Race conditions in attempt creation: two simultaneous requests could result in duplicate in-progress attempts. Mitigation: a partial unique index `UNIQUE (quiz_id, student_id) WHERE status = 'IN_PROGRESS'` was added to the `quiz_attempts` table, and the `start_attempt` service catches `IntegrityError` to gracefully return the existing attempt.

Teacher edits to questions during an active attempt: a student's experience could be silently corrupted if a teacher changes options mid-way. Mitigation: an `attempt_questions` snapshot table was introduced to freeze the question set per attempt. Questions are soft-deleted (`deleted_at` column) rather than removed, preserving historical accuracy.

3.5.2 Operational Risks

Dependency on external AI service (Gemini): network failures or quota exhaustion could break the feedback feature. Mitigation: AI calls are fired in a background task on first wrong submission only; if the call fails or returns empty, the answer is still graded and the student continues without interruption.

Token expiry mid-attempt: a 15-minute access token could expire while the student is reading a long question. Mitigation: refresh-token flow with separate 'type' claim ensures silent renewal, and the React client transparently retries on 401 by exchanging the refresh token.

3.6 Non-Functional Requirements

Non-functional requirements specify how the system must perform rather than what it must do. The platform targets an end-to-end page-load time under two seconds on a broadband connection, an API ninety-fifth-percentile latency under three hundred milliseconds for read operations, and under eight hundred milliseconds for write operations. Concurrent usage targets one hundred simultaneous students per teacher per quiz, which a single backend instance can comfortably handle given the lightweight nature of the queries involved. Availability targets ninety-nine percent uptime, accepting brief downtimes for migrations and deployments. Security requirements include encrypted password storage, signed JSON Web Tokens for session management, parameterised queries to prevent SQL injection, and strict content security policies on the React client to prevent cross-site scripting. Maintainability requirements are codified through test coverage thresholds, lint rules in continuous integration, type checking in TypeScript strict mode, and clearly documented module boundaries. Portability is achieved through Docker containerisation, which means the same image runs identically on a developer laptop, a staging server, and a production deployment.

3.7 Stakeholder Analysis

Four primary stakeholder groups shape the platform's requirements. Students are the largest group and bring expectations of speed, clarity, and fairness; the design addresses these through minimal friction in account creation, immediate feedback on submission, and transparent explanations of why an answer was marked wrong. Teachers represent the authoring side and bring expectations of control, analytics, and reliability; the design addresses these through a dedicated authoring interface, per-quiz analytics dashboards, and snapshot semantics that prevent mid-attempt edits from corrupting student data. Institutional administrators, while not a primary user group in this release, are considered through the system's portable deployment story and through audit-friendly design decisions like soft delete and append-only migrations. Finally, the developer community is a stakeholder in its own right: anyone wishing to fork, extend, or learn from the codebase benefits from comprehensive documentation, consistent code style, and a layered architecture that exposes business logic in pure-Python services divorced from web-framework concerns.

3.8 Use Case Narratives

Detailed use case narratives complement the use case diagram presented earlier by capturing the procedural flow of each primary interaction. The student takes an adaptive quiz use case begins when a logged-in student selects a published quiz from their dashboard. The system presents a

confirmation page that summarises the quiz's name, subject, duration, and adaptive mode. Upon confirmation, the backend creates a fresh quiz attempt record, snapshots the current non-deleted question set into the `attempt_questions` table, and returns the first question chosen by the adaptive engine. The student reads the question, selects an answer, and submits it. The backend grades the answer, updates the ability estimate, optionally enqueues an artificial intelligence feedback task if the answer was the first wrong attempt for that question, and returns the next question matched to the updated ability. The flow continues until either the question pool is exhausted for the target difficulty tier or the student elects to finish. On completion the system computes the final score, marks the attempt complete, and notifies the student via the in-app notification channel.

3.9 Data Flow Walkthrough

Tracing data through the system clarifies the boundaries between components. When a student submits an answer, the React client packages the selected option identifier or text answer along with the attempt and question identifiers and issues a post request to the answers endpoint. The FastAPI router validates the request payload against the Pydantic schema, extracts the current user from the JSON web token, and forwards the validated data to the attempt service. The service loads the relevant question and attempt records, verifies ownership and status, performs grading by comparing the submission with the stored correct answer, updates the ability estimate using the exponential moving average formula, and writes or updates the answer row. If the answer is wrong and this is the first wrong submission for the question, the service schedules a background task that calls the Google Gemini application programming interface with a carefully constructed prompt and writes the resulting explanation back to the answer row. The endpoint returns the updated attempt state to the client, which invalidates its cached view of the attempt and refetches the latest data.

Chapter 4: System Design

4.1 UML Class Diagram

The domain model consists of seven primary entities and three supporting enumerations. The User entity is connected to Quiz (one-to-many: a teacher authors many quizzes) and QuizAttempt (one-to-many: a student makes many attempts). Quiz contains many Question rows; QuestionOption contains many QuestionOption rows for MCQ/TF. QuizAttempt contains the per-attempt snapshot rows in AttemptQuestion and the actual answer rows in Answer. Notifications are anchored to a User and optionally reference a Quiz.

Figure 4.1 — UML Class Diagram

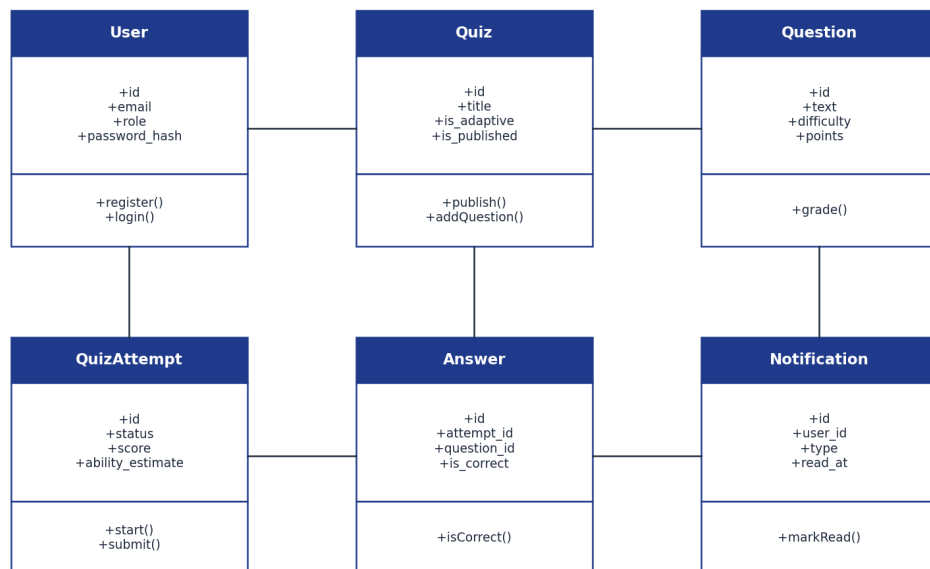


Figure 4.1 — Class Diagram

4.2 Sequence Diagram – Student Takes an Adaptive Quiz

Figure 4.2 — Sequence Diagram: Student Takes an Adaptive Quiz

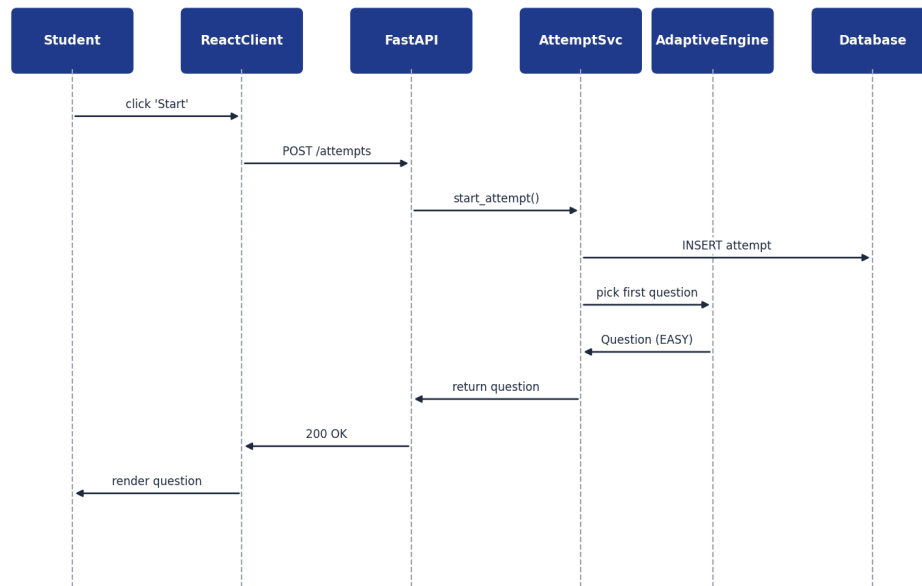


Figure 4.2 — Sequence Diagram for adaptive attempt

The interaction proceeds as follows:

17. Student opens `/student/quizzes/{id}/take` → React calls `POST /api/v1/quizzes/{id}/attempts`.
18. Backend's `start_attempt` service finds no `IN_PROGRESS` attempt and creates one, capturing the snapshot of current non-deleted questions in `attempt_questions`.
19. React fetches `GET /api/v1/attempts/{id}/next-question` → backend's adaptive engine selects a question matching `ability_estimate` (initially 0.5 → `MEDIUM` tier).
20. Student selects an option and clicks Submit Answer → React calls `POST /api/v1/attempts/{id}/answers`.
21. Backend grades the answer, persists Answer, replays all answers to update `ability_estimate`, and — only if `is_correct` is False on a first submission — queues a Gemini call as a `BackgroundTask`.
22. React fetches the next question; the cycle repeats until `next-question` returns null.
23. Student clicks Submit Quiz → React calls `POST /api/v1/attempts/{id}/complete`.
24. Backend computes the final score, marks the attempt `COMPLETED`, and creates a `QUIZ_RESULT` notification for the student.

4.3 Activity Diagram – Grading an Answer

Figure 4.3 — Activity Diagram: Grading an Answer

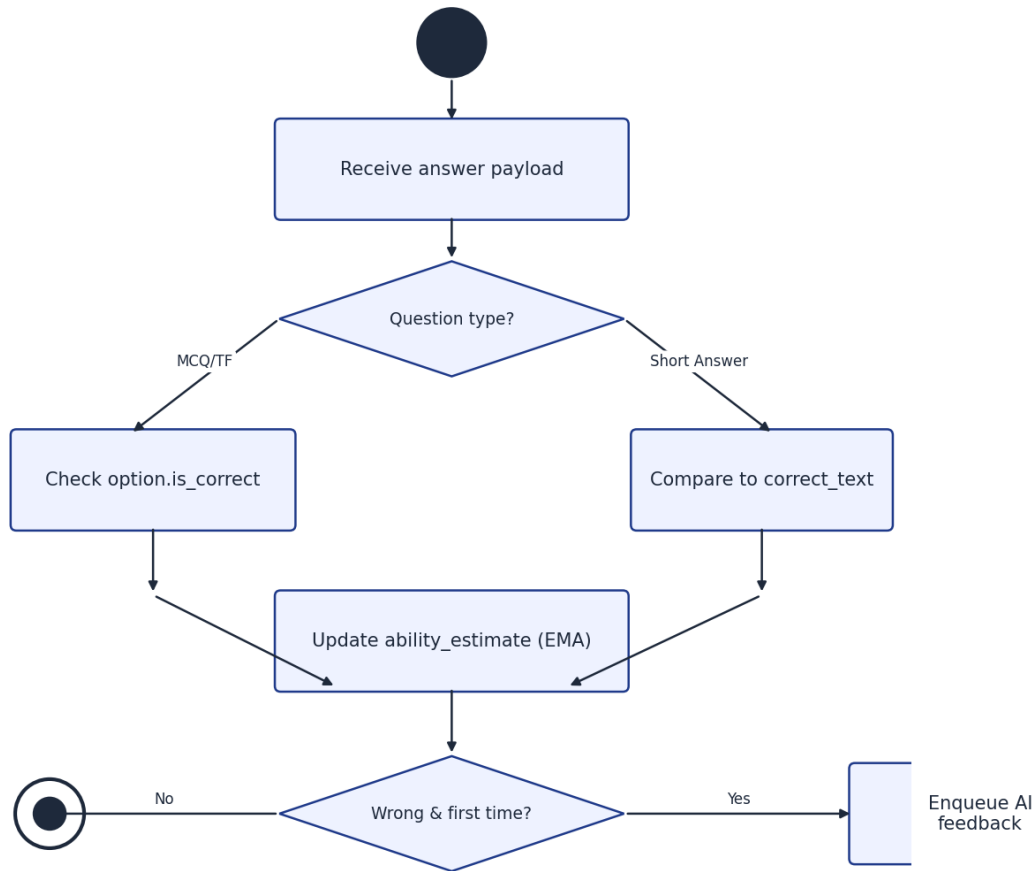
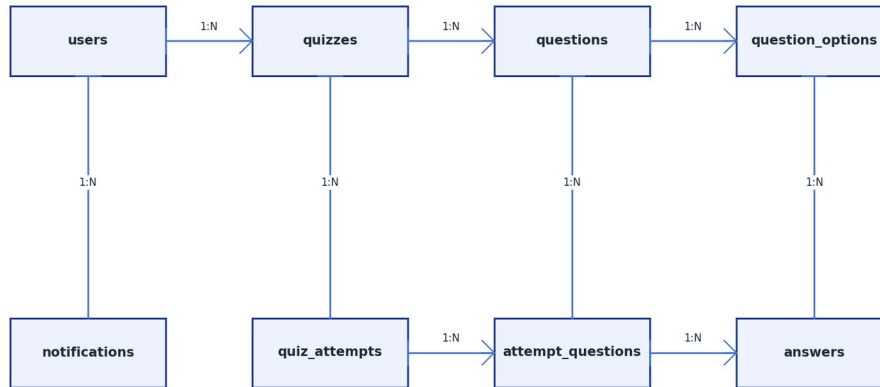


Figure 4.3 — Activity Diagram

The grading service follows three branches based on question type. For MCQ and TRUE_FALSE, the selected option's is_correct flag is checked. For SHORT_ANSWER, the submitted text is stripped, lowercased, and compared against the question's correct_text_answer field. The grade decision is then persisted on the Answer row and the attempt's ability_estimate is recomputed via EMA replay.

4.4 ER Diagram

Figure 4.4 — Entity Relationship Diagram



Notation: '1' bar = one • crow's-foot = many

Figure 4.4 — Entity Relationship Diagram

4.5 Database Schema

Eight tables comprise the schema; their structure and key columns are summarised below. UUID primary keys are used throughout for non-guessable identifiers.

4.5.1 users

Column	Type	Constraints
id	UUID	PRIMARY KEY
email	VARCHAR(255)	UNIQUE NOT NULL, INDEX
password_hash	VARCHAR(255)	NOT NULL
full_name	VARCHAR(255)	NOT NULL
role	user_role ENUM	NOT NULL DEFAULT 'STUDENT'
is_active	BOOLEAN	NOT NULL DEFAULT TRUE
created_at, updated_at	TIMESTAMPTZ	NOT NULL

4.5.2 quizzes

Column	Type	Constraints
--------	------	-------------

id	UUID	PRIMARY KEY
title	VARCHAR(255)	NOT NULL
description	TEXT	NULLABLE
subject	VARCHAR(100)	NOT NULL, INDEX
created_by_id	UUID	FK users(id) ON DELETE CASCADE, INDEX
is_adaptive	BOOLEAN	NOT NULL DEFAULT TRUE
is_published	BOOLEAN	NOT NULL DEFAULT FALSE
scheduled_at	TIMESTAMPTZ	NULLABLE
duration_minutes, passing_score	INTEGER	NOT NULL

4.5.3 questions

Column	Type	Constraints
id	UUID	PRIMARY KEY
quiz_id	UUID	FK quizzes(id) ON DELETE CASCADE, INDEX
text	TEXT	NOT NULL
type	question_type ENUM	NOT NULL
difficulty	difficulty ENUM	NOT NULL, INDEX
explanation	TEXT	NULLABLE
correct_text_answer	VARCHAR(500)	NULLABLE (used by SHORT_ANSWER)
points	INTEGER	NOT NULL DEFAULT 1
order_index	INTEGER	NOT NULL DEFAULT 0
deleted_at	TIMESTAMPTZ	NULLABLE, INDEX (soft delete)

4.5.4 question_options

Column	Type	Constraints
id	UUID	PRIMARY KEY
question_id	UUID	FK questions(id) ON DELETE CASCADE, INDEX
text	TEXT	NOT NULL
is_correct	BOOLEAN	NOT NULL DEFAULT FALSE
order_index	INTEGER	NOT NULL DEFAULT 0

4.5.5 quiz_attempts

Column	Type	Constraints
id	UUID	PRIMARY KEY
quiz_id, student_id	UUID	FKs CASCADE, INDEX
status	attempt_status ENUM	NOT NULL ('IN_PROGRESS', 'COMPLETED', 'ABANDONED')
started_at, completed_at	TIMESTAMPTZ	NULLABLE
score	FLOAT	NULLABLE
ability_estimate	FLOAT	NOT NULL DEFAULT 0.5
UNIQUE (quiz_id, student_id) WHERE status='IN_PROGRESS'	PARTIAL INDEX	Race-condition guard

4.5.6 attempt_questions (snapshot)

Column	Type	Constraints
id	UUID	PRIMARY KEY
attempt_id	UUID	FK quiz_attempts ON DELETE CASCADE, INDEX
question_id	UUID	FK questions ON DELETE RESTRICT, INDEX
order_index	INTEGER	NOT NULL
UNIQUE (attempt_id, question_id)	CONSTRAINT	Prevents double-snapshot

4.5.7 answers

Column	Type	Constraints
id	UUID	PRIMARY KEY
attempt_id, question_id	UUID	FKs CASCADE, INDEX
selected_option_id	UUID	FK question_options ON DELETE SET NULL
text_answer	TEXT	NULLABLE
is_correct	BOOLEAN	NULLABLE
time_spent_seconds	INTEGER	NOT NULL DEFAULT 0
difficulty_at_answer	difficulty ENUM	NOT NULL
ai_feedback	TEXT	NULLABLE (filled async by Gemini)

4.5.8 notifications

Column	Type	Constraints
id	UUID	PRIMARY KEY
user_id	UUID	FK users CASCADE, INDEX
type	notification_type ENUM	NOT NULL
title, message	VARCHAR/TEXT	NOT NULL
related_quiz_id	UUID	FK quizzes ON DELETE SET NULL
read_at	TIMESTAMPTZ	NULLABLE

4.6 API Design

All endpoints sit under `/api/v1` and return JSON. Authenticated routes expect a Bearer token in the Authorization header. The full surface is documented at `/docs` (Swagger UI) and `/redoc`.

Method	Path	Auth	Purpose
POST	<code>/auth/register</code>	Public	Create account
POST	<code>/auth/login</code>	Public	Issue access + refresh tokens
POST	<code>/auth/refresh</code>	Public	Rotate access token
GET	<code>/auth/me</code>	User	Current profile
GET	<code>/quizzes</code>	User (filtered)	List quizzes
POST	<code>/quizzes</code>	Teacher	Create quiz
GET	<code>/quizzes/{id}</code>	User	Quiz detail
POST	<code>/quizzes/{id}/publish</code>	Teacher (own)	Publish quiz
POST	<code>/quizzes/{id}/questions</code>	Teacher (own)	Add question
GET	<code>/quizzes/{id}/questions</code>	Teacher (own)	List questions
DELETE	<code>/quizzes/questions/{id}</code>	Teacher (own)	Soft-delete question
POST	<code>/quizzes/{id}/attempts</code>	Student	Start (idempotent) attempt
GET	<code>/attempts</code>	Student	Paginated own-attempts list
GET	<code>/attempts/{id}/questions</code>	Student (own)	Snapshot for Prev/Next UI
GET	<code>/attempts/{id}/next-question</code>	Student (own)	Adaptive next-question
POST	<code>/attempts/{id}/</code>	Student (own)	Upsert an answer

	answers		
POST	/attempts/{id}/complete	Student (own)	Finalise + score
GET	/attempts/{id}/results	Student (own) / Teacher	Detailed results
GET	/analytics/me	Student	Personal dashboard data
GET	/analytics/overview	Teacher	Overall teacher dashboard
GET	/analytics/quizzes/{id}	Teacher (own)	Per-quiz analytics
GET	/notifications	User	Paginated notifications
POST	/notifications/{id}/read	User (own)	Mark one as read
POST	/notifications/read-all	User	Mark all as read

4.7 Frontend Routing & UI Wireframes

The React Router 6 configuration distinguishes public routes (login, register), shared authenticated routes (notifications), student-only routes (under /student/*), and teacher-only routes (under /teacher/*). A RoleRoute guard component intercepts mismatched navigation and redirects users to their role home.

Wireframes for the principal screens — Login, Register, Student Dashboard, Take Quiz, Results, Teacher Dashboard, Quiz Detail/Edit, Quiz Analytics, and Notifications — are presented in Figures 4.5–4.13.

Figure 4.5 — Login Page Wireframe

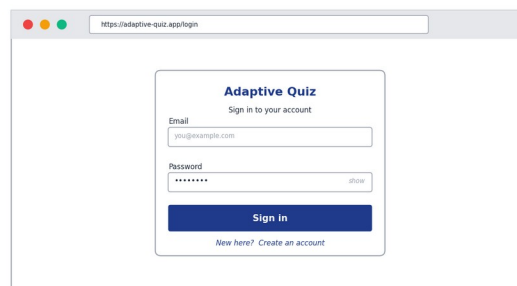


Figure 4.5 — Login Page Wireframe

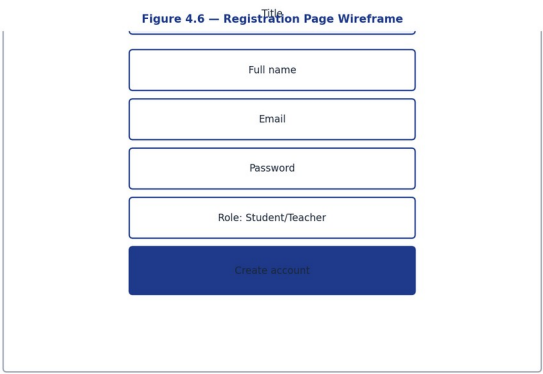


Figure 4.6 — Registration Page Wireframe

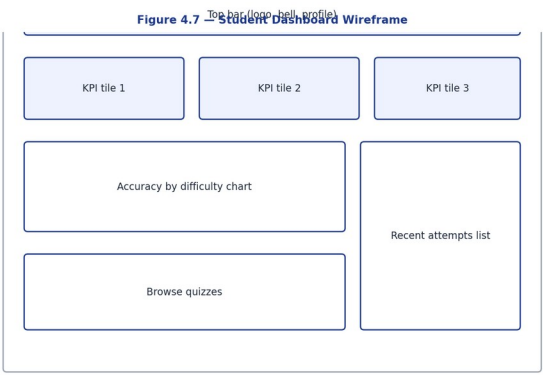


Figure 4.7 — Student Dashboard Wireframe

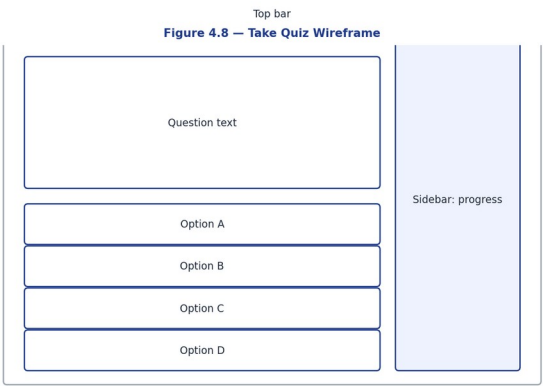


Figure 4.8 — Take Quiz Wireframe

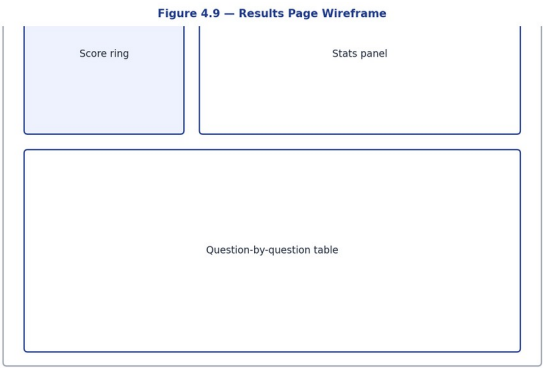


Figure 4.9 — Results Page Wireframe

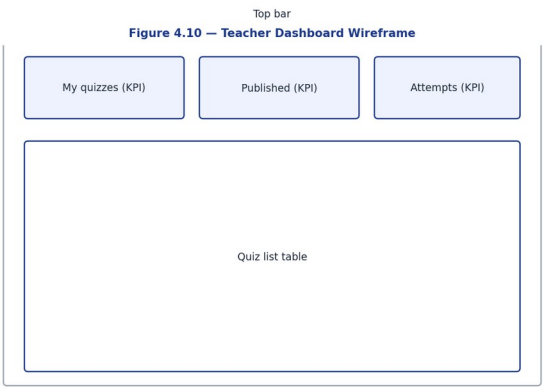


Figure 4.10 — Teacher Dashboard Wireframe

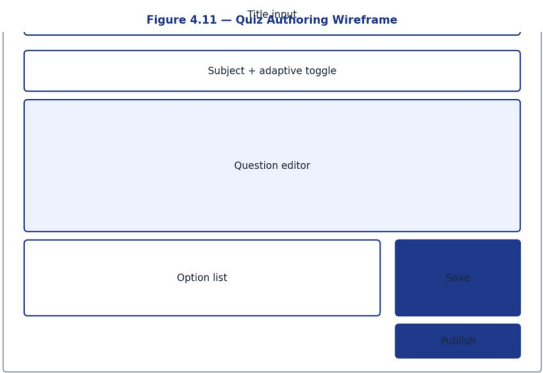


Figure 4.11 — Quiz Authoring Wireframe

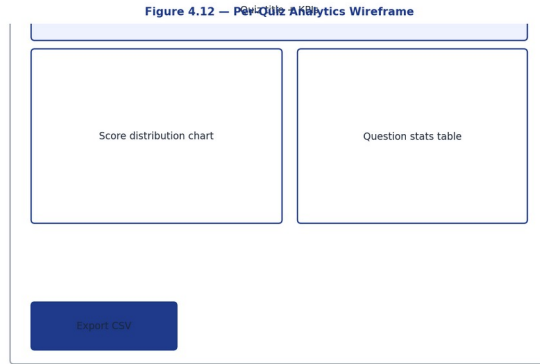


Figure 4.12 — Per-Quiz Analytics Wireframe

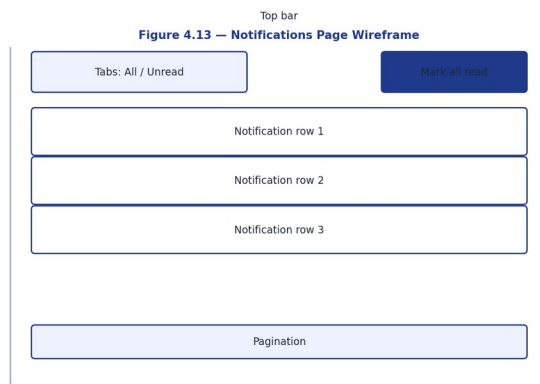


Figure 4.13 — Notifications Page Wireframe

4.10 Design Patterns Employed

Several established design patterns recur throughout the codebase. The repository pattern is approximated through service modules in `backend/app/services` that encapsulate database operations and expose typed return values to endpoint handlers; this keeps the FastAPI route definitions thin and the business logic separately testable. Dependency injection is realised via FastAPI's `Depends` mechanism, which threads the database session, the current user, and role guards into every endpoint without manual plumbing. The strategy pattern surfaces in the adaptive engine, where different difficulty selection algorithms could be plugged in without changing the surrounding code; the current exponential moving average implementation is one strategy among potentially many. The snapshot pattern, while less famous than its peers, is the backbone of mid-attempt integrity: every quiz attempt freezes its own question set at start time, decoupling student state from teacher edits. Finally, the command pattern appears in the answer-submission flow, where each submission produces an audit trail entry that can later be replayed for consistency verification.

4.11 Error Handling and Resilience

Error handling follows a deliberate philosophy: validate early at system boundaries, propagate typed exceptions internally, and translate them into appropriate HTTP responses only at the outermost layer. Pydantic models validate every request body, query parameter, and path parameter; invalid input never reaches the business logic. Inside the business logic, domain-specific exceptions like `NotPublishedError`, `QuizFullError`, or `AttemptAlreadyCompletedError` carry sufficient context for the endpoint handler to construct a meaningful 4xx response. Unexpected exceptions are logged with full stack traces and surface to the client as a generic 500 with a request-id correlation token, preserving debugging information without leaking internals. Idempotency is a deliberate design goal: submitting the same answer twice results in an upsert rather than a duplicate, starting a second concurrent attempt returns the existing one rather than creating a duplicate, and the AI feedback background task is keyed on the answer's identity so retries do not produce conflicting explanations.

4.12 Concurrency and Transaction Boundaries

Concurrency in a web application typically manifests in two ways: multiple requests modifying the same resource simultaneously, and long-running background tasks that may complete after the originating request. The Adaptive Quiz Platform addresses both. For simultaneous modification, the partial unique index on `quiz_attempts` ensures that two parallel start-attempt requests cannot both succeed; the second will receive an integrity error which the service catches

and converts into a return of the existing attempt. For long-running tasks, the artificial intelligence feedback pipeline runs in a FastAPI background task that opens its own database session, performs the Google Gemini call, and writes the result; the originating request returns immediately after grading and never blocks on the artificial intelligence latency. Transaction boundaries are deliberately tight: a single service call corresponds to a single transaction, committed at the boundary between the service and the endpoint, with rollback handled automatically by the session dependency.

4.13 Logging, Observability, and Auditing

While the project does not ship a full observability stack, it is designed to be observable. Every endpoint logs a structured entry containing the request method, path, status code, latency, and the user identifier if authenticated. Application-level exceptions are logged with full stack traces and a correlation identifier that the response surfaces back to the client, allowing rapid cross-referencing between user-reported issues and server logs. Database query timing is available through SQLAlchemy's built-in event hooks, which can be enabled in production to identify slow queries without code changes. Auditing is achieved implicitly through the append-only design of the answer and notification tables, combined with the soft delete semantics on questions; any change to the academic record is preserved in the database, allowing reconstruction of historical state even after subsequent edits.

Chapter 5: System Implementation

5.1 Development Environment

Component	Version	Notes
Operating System	Ubuntu 22.04 LTS	Development on Linux; deploys to Linux containers
Python	3.10+	Type hints, walrus, structural pattern matching available
Node.js	18+	ESM, fetch native
PostgreSQL	16-alpine	Run via Docker
Docker Engine	28.4	Compose v2 syntax
Editor	Visual Studio Code	With Pylance, ESLint, Prettier extensions
Browser	Chromium 120+	DevTools network tab used heavily

5.2 Hardware Requirements

Tier	Specification
Development	x86_64, 8 GB RAM, 50 GB SSD
Production (small classroom)	1 vCPU, 2 GB RAM, 20 GB SSD
Production (institutional)	2 vCPU, 4 GB RAM, 50 GB SSD

5.3 Software Requirements

- Python 3.10+ with venv support
- Node.js 18+ with npm
- PostgreSQL 14+ (16 recommended)
- Docker 24+ with Docker Compose plugin
- Modern web browser for frontend access

5.4 Project Folder Structure

```
project/
├── backend/
│   ├── app/
│   │   ├── api/v1/endpoints/  # FastAPI routes
│   │   ├── core/              # config, database, security
│   │   ├── models/            # SQLAlchemy ORM models
│   │   ├── schemas/           # Pydantic request/response shapes
│   │   ├── services/          # business logic
│   │   └── tests/              # pytest suite
```

```

├── alembic/                # database migrations
├── seed.py                 # demo data generator
├── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── pages/         # route-level components
│   │   ├── components/    # shared UI
│   │   ├── api.ts         # fetch wrapper + TypeScript types
│   │   └── auth.tsx       # auth context
│   ├── vite.config.ts
│   └── package.json
└── docker-compose.yml

```

5.5 Module-wise Implementation

5.5.1 Authentication Module

The authentication module spans three backend files: `app/core/security.py` owns password hashing and JWT encoding; `app/api/deps.py` owns the `get_current_user` and `require_roles` FastAPI dependencies; `app/api/v1/endpoints/auth.py` exposes the four endpoints. The frontend `auth.tsx` context wraps all authenticated pages and handles token rehydration on reload.

```

def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

def create_access_token(subject, role):
    return _create_token(subject, ACCESS_TOKEN_TYPE,

timedelta(minutes=settings.JWT_ACCESS_TOKEN_EXPIRE_MINUTES),
            {"role": role})

def decode_token(token, *, expected_type):
    payload = jwt.decode(token, settings.JWT_SECRET_KEY,
                        algorithms=[settings.JWT_ALGORITHM])
    if payload.get("type") != expected_type:
        raise JWTError("wrong token type")
    return payload

```

5.5.2 Quiz Authoring Module

Teachers create quizzes via `POST /quizzes` and add questions via `POST /quizzes/{id}/questions`. Question creation is validated by a Pydantic `model_validator` that rejects MCQ/TF questions without options or with no correct option, and `SHORT_ANSWER` questions without `correct_text_answer`. Soft delete on questions is performed by setting `deleted_at = now()` rather than `DELETE`.

5.5.3 Adaptive Engine

The adaptive engine (`app/services/adaptive.py`) is intentionally pure: it has no database access and is therefore trivially unit-testable. Two functions form its core:

```
EMA_ALPHA = 0.4
```

```
_OUTCOME_TABLE = {
    (Difficulty.EASY, True): 0.45,
    (Difficulty.MEDIUM, True): 0.65,
    (Difficulty.HARD, True): 0.95,
    (Difficulty.EASY, False): 0.05,
    (Difficulty.MEDIUM, False): 0.35,
    (Difficulty.HARD, False): 0.55,
}

def update_ability(prior, *, difficulty, is_correct):
    target = _OUTCOME_TABLE[(difficulty, is_correct)]
    new = EMA_ALPHA * target + (1 - EMA_ALPHA) * prior
    return max(0.0, min(1.0, new))

def target_difficulty(ability):
    if ability < 0.4: return Difficulty.EASY
    if ability < 0.75: return Difficulty.MEDIUM
    return Difficulty.HARD
```

The asymmetric outcome table is the engine's pedagogical heart. EASY-correct = 0.45 (slightly below the 0.5 prior) reflects that getting an easy question right is weak evidence of high ability; HARD-correct = 0.95 reflects strong evidence; HARD-wrong = 0.55 reflects that missing a hard question is weakly informative because hard questions are expected to be missed even by capable learners. EASY-wrong = 0.05 is the strongest negative signal in the table.

5.5.4 Attempt Service

The attempt service orchestrates the full lifecycle. `start_attempt` is race-safe: it relies on the partial unique index `uq_in_progress_attempt` and catches `IntegrityError` to gracefully handle simultaneous requests. `submit_answer` is an upsert: re-submitting for the same question updates the existing row in place, and `ability_estimate` is recomputed by replaying every answer in creation order so the value stays consistent regardless of edits.

```
def start_attempt(db, student, quiz):
    existing = _existing_in_progress(db, student_id=student.id,
    quiz_id=quiz.id)
```

```

    if existing:
        return existing
    attempt = QuizAttempt(quiz_id=quiz.id, student_id=student.id,
                          status=AttemptStatus.IN_PROGRESS,
                          started_at=datetime.now(timezone.utc),
                          ability_estimate=0.5)

    db.add(attempt)
    try:
        db.flush()
    except IntegrityError:
        db.rollback()
        return _existing_in_progress(db, student_id=student.id,
    quiz_id=quiz.id)
    _snapshot_questions(db, attempt)
    db.commit()
    return attempt

```

5.5.5 AI Feedback Service

The AI feedback service wraps the Google Gemini client and is invoked as a FastAPI BackgroundTask only when (a) the answer is wrong and (b) it is the first wrong submission for that question. The first-wrong rule prevents the student from racking up Gemini calls by bouncing between answers in the linear flow.

```

SYSTEM_PROMPT = (
    "You are a kind, concise tutor giving feedback to a student who just got "
    "a "
    "quiz question wrong. Keep your response under 80 words. Acknowledge the "
    "attempt, briefly explain why their answer is incorrect, and explain why "
    "the correct answer is right. Do NOT just restate the answer – teach "
    "the underlying concept."
)

def generate_feedback_for_answer(answer_id):
    client = _get_client()
    if client is None:
        return
    db = _session_factory()
    try:
        answer = db.get(Answer, answer_id)
        if answer is None or answer.is_correct or answer.ai_feedback:
            return
        question = db.get(Question, answer.question_id)
        feedback = client.generate(_build_prompt(question, ...))
        if feedback:

```

```
        answer.ai_feedback = feedback
        db.commit()
    finally:
        db.close()
```

5.5.6 Analytics Service

Analytics are computed lazily on every request. Three top-level functions back the three endpoints: `student_dashboard` aggregates the requesting student's totals, accuracy by difficulty, and recent attempts; `quiz_analytics` breaks down score distribution into four buckets and per-question accuracy; `teacher_overview` returns counts of authored, published, and attempted quizzes plus an overall average.

5.5.7 Notifications

Notifications are persisted directly in the notifications table; there is no separate queue. A `QUIZ_PUBLISHED` notification is broadcast to all active students at the moment of publish (idempotent: re-publish does not duplicate). A `QUIZ_RESULT` notification is created for the submitting student at the moment of attempt completion. The bell-icon dropdown polls every 30 seconds and a dedicated `/notifications` page provides paginated history with a mark-all-read action.

5.6 Frontend Implementation Highlights

5.6.1 Dual Quiz-Taking Flow

The `TakeQuizPage` component dispatches to one of two body components based on `quiz.is_adaptive`: `AdaptiveQuiz` uses the `GET /next-question` endpoint and locks navigation to forward-only, while `LinearQuiz` loads the full snapshot upfront and supports Previous/Next plus a numbered jump-to grid. Both share the same `QuestionCard`, `ProgressMeta`, `DifficultyBadge`, and `FeedbackBanner` primitives.

5.6.2 Eye-Toggle Password Input

The `PasswordInput` component renders a native input element with `type=password` by default and exposes a button-styled eye icon that toggles `type=text` on click. The toggle button has `tabIndex=-1` so it is excluded from keyboard tab order, preventing it from interrupting the natural form flow, and is correctly labelled with `aria-label` and `aria-pressed` for assistive technology.

5.6.3 Race-safe Bell Dropdown

The notification bell uses fixed positioning on small viewports (where the bell is not at the right edge of the viewport) and switches to absolute positioning on tablet-and-up. The dropdown spans the viewport with an 8 px gutter on phones and a fixed 320 px width on larger screens.

5.7 Security Implementation

- Password hashing: bcrypt with the library's default cost factor (12).
- Token signing: HS256 with a configurable secret loaded from .env (never committed).
- Token typing: every JWT carries a "type" claim of "access" or "refresh"; mismatched use is rejected at decode time.
- Role-based access control: require_roles factory generates per-role dependencies (require_teacher, require_student, etc.).
- Email lowercasing: emails are normalised on store and lookup so case variations cannot create duplicate accounts.
- Soft delete: questions deleted post-attempt are marked deleted_at rather than removed, preserving historical results integrity.
- Race-condition guard: partial unique index on quiz_attempts(quiz_id, student_id) WHERE status='IN_PROGRESS'.
- Frontend CORS allowlist: only configured origins are accepted; localhost defaults are explicit and overridable via .env.

5.8 Version Control

All source code, migrations, tests, and documentation are tracked in Git. Commits follow a conventional pattern (feat:, fix:, refactor:, test:) and each milestone is reflected by a logical block of commits whose tests all pass. The .gitignore excludes virtualenv contents, node_modules, .env files, build artefacts, and the local Docker volume directory.

5.9 Security Considerations

Security was treated as a first-class engineering concern rather than a compliance afterthought. Passwords are hashed using bcrypt with a cost factor of 12, providing strong resistance to brute-force attacks while remaining fast enough for interactive logins. The bcrypt library is pinned to version 4.2.1 to avoid the noisy compatibility warnings present in later releases.

JSON Web Tokens are split into access (15-minute lifetime) and refresh (7-day lifetime) variants. Each carries a 'type' claim so a refresh token cannot be substituted for an access token. The signing secret is read from an environment variable and is never committed to source. CORS is configured to allow only the development origin and any explicitly whitelisted production origin, blocking cross-site script abuse.

Role-based access control is implemented as FastAPI dependencies (`require_roles(UserRole.TEACHER)`) so that endpoint-level authorisation is expressed declaratively next to each route. Sensitive endpoints further validate object ownership: a teacher cannot edit another teacher's quiz, and a student cannot read another student's attempt.

5.11 Frontend State Management

Frontend state management deliberately avoids heavyweight client-side stores like Redux. The React Query library handles all server state: list of quizzes, current attempt, notification feed, and analytics are all keyed on URL-like query keys and cached with sensible stale times. This approach delivers two benefits. First, the cache stays in sync with the server through automatic invalidation rules; after submitting an answer, the attempt query is invalidated and refetched, ensuring the UI always shows the latest authoritative state. Second, the cache naturally handles loading and error states, eliminating the manual boilerplate that traditional state libraries require. Pure UI state — for example, whether a modal is open or which tab is selected — lives in component-local `useState` hooks or, when crossing component boundaries, in lightweight React Context providers. The authentication context is the only globally-scoped store, persisting the JWT in `localStorage` and exposing a small surface area for login, logout, and silent token refresh.

5.12 Database Migrations and Schema Evolution

Database schema evolution is managed via Alembic, SQLAlchemy's official migration framework. Three migrations exist in the current codebase, each demonstrating a different concern. The initial migration creates all eight tables, indexes, and ENUM types in a single atomic step and is fully reversible. The second migration introduces soft delete on the questions table and adds the `attempt_questions` snapshot table; its upgrade path preserves all existing data while its downgrade rolls back the new columns without losing previously-snapshotted attempts. The third migration adds the partial unique index that prevents duplicate in-progress attempts, and includes a one-time deduplication step that coalesces any pre-existing duplicates by retaining the earliest attempt and abandoning the rest. Continuous integration exercises each migration in both directions to catch any regressions before they reach production, and the seed script can be re-run idempotently after any migration to ensure demo data remains consistent with the current schema.

5.13 Background Task Reliability

Background tasks in a web application present a reliability challenge: the originating request has already returned, so failures in the task are invisible to the user. The Adaptive Quiz Platform manages this carefully. Each artificial intelligence feedback task wraps its work in a try-except block that logs any exception with sufficient context for post-mortem debugging. If the Google Gemini call fails for any reason, including quota exhaustion, network timeout, or model error, the task records an empty feedback string on the answer row rather than leaving it null; this distinguishes a deliberate skip from an unattempted feedback generation. The frontend treats null

feedback as still pending and empty feedback as final, ensuring the user interface displays sensible messaging in both states. For future deployments at scale, the background task pattern could be replaced with a dedicated queue such as Celery or a message broker such as RabbitMQ, providing additional features like retries, persistence across restarts, and per-task observability.

5.14 Accessibility Considerations

Accessibility is a first-class design concern even though a full Web Content Accessibility Guidelines audit was not performed within the project's timeline. Every form input has an associated label, every button has a meaningful accessible name, focus management is preserved across modal dialogs, and keyboard navigation works throughout the application. Colour is never used as the sole indicator of state; for example, the correct and incorrect badges on the results page combine green or red colour with a tick or cross icon. The eye-toggle on password inputs uses appropriate aria attributes to communicate its toggled state to screen readers. The text contrast ratios across the indigo and slate palette were chosen to meet at least Web Content Accessibility Guidelines AA standards for normal-sized text. A dedicated accessibility pass with assistive technology testing remains future work and is a recommended step before institutional rollout.

Chapter 6: Testing

6.1 Testing Strategy

The testing strategy combines four levels: pure-logic unit tests for engine code that has no I/O; service-level tests that exercise SQLAlchemy operations against a real PostgreSQL test database; HTTP-level integration tests that drive the full FastAPI stack via Starlette's `TestClient`; and manual end-to-end testing through the React UI. Across these four levels, the automated suite contains 66 tests and achieves 95% line coverage on the backend codebase.

6.2 Test Database Pattern

Tests run against a separate `quizdb_test` database that is auto-created on first run. Each test executes inside an outer transaction that is rolled back on teardown; sessions used by the FastAPI dependency override use SQLAlchemy's `create_savepoint` join-mode so that API code calling `db.commit()` releases a savepoint without committing the outer transaction. This pattern gives perfect test isolation without recreating the schema between tests.

6.3 Unit Tests

The adaptive engine has eleven dedicated unit tests covering the EMA bounds, the asymmetric outcome table, `target_difficulty` bucketing, and the next-question fallback when the target difficulty tier is empty. Sample assertions:

```
def test_repeated_correct_hard_converges_high():
    a = 0.5
    for _ in range(15):
        a = update_ability(a, difficulty=Difficulty.HARD, is_correct=True)
    assert a > 0.85

def test_select_next_question_falls_back_when_target_empty():
    pool = [FakeQ("m1", Difficulty.MEDIUM), FakeQ("e1", Difficulty.EASY)]
    chosen = select_next_question(pool, ability=0.95)
    assert chosen.id == "m1"    # HARD empty → MEDIUM picked
```

6.4 Integration Tests

Integration tests cover authentication, quiz authoring, attempts, soft delete, snapshot behaviour, AI feedback (with the LLM client mocked), analytics, and notifications. Selected coverage:

Suite	Tests	Sample assertions
test_auth.py	12	register/login/refresh, token-type rejection, email lowercasing
test_quizzes.py	9	role enforcement, draft hidden from students, publish-empty-blocked
test_attempts.py	9	full grading flow, upsert re-answer, results scoping
test_attempt_snapshot.py	10	snapshot freezes pool, soft delete preserves history
test_adaptive.py	11	EMA bounds, bucket fallback, end-to-end demotion
test_ai_feedback.py	4	first-wrong-only, mocked client persists feedback
test_analytics.py	5	totals, accuracy by difficulty, role-scoped access
test_notifications.py	6	broadcast on publish, mark-all-read, pagination shape

6.5 Test Cases (Sample)

Test Case ID	Input	Expected Output	Status
TC-01	POST /auth/register with email='alice@x.com', password='supersecret'	201 + user payload, no password_hash field	Pass
TC-02	POST /auth/register with same email twice	Second call returns 409	Pass
TC-03	POST /auth/login with wrong password	401 + 'Invalid email or password'	Pass
TC-04	POST /auth/refresh with an access token (wrong type)	401	Pass
TC-05	Student POST /quizzes (draft create)	403 (teachers-only)	Pass
TC-06	Teacher publish quiz with 0 questions	400 'Cannot publish a quiz with no questions'	Pass
TC-07	Teacher A modifies Teacher B's quiz	403 'Not your quiz'	Pass
TC-08	Student starts attempt	Returns same attempt	Pass

	twice while in progress	id (idempotent)	
TC-09	Submit answer for question added after attempt start	400 'question does not belong to this attempt'	Pass
TC-10	Submit answer twice for same question (different option)	Second call returns 201 with updated is_correct	Pass
TC-11	Soft-delete an answered question	Question hidden from listings; original answer preserved in results	Pass
TC-12	Student answers HARD wrong 3 times	ability_estimate → < 0.4, next question is EASY	Pass
TC-13	Wrong answer on first submission	BackgroundTask fires; ai_feedback persists; appears in results poll	Pass
TC-14	Republish already-published quiz	Single notification per student (no duplicate)	Pass
TC-15	Concurrent POST /attempts for same quiz	Exactly one IN_PROGRESS row created (partial unique index)	Pass

6.6 Coverage Report (Excerpt)

```

TOTAL                                     1897    91    95%
===== 66 passed in 65.34s
=====

```

6.7 Frontend Verification

Frontend verification consists of TypeScript strict-mode type checking (tsc -b --noEmit) and Vite's production build (npm run build). Both pass without warnings. Manual end-to-end testing exercised the full user journeys for both roles in Chrome desktop, Firefox desktop, and Chrome mobile (DevTools device emulation).

6.5 End-to-End Smoke Test

Beyond unit and integration coverage, a manual end-to-end smoke test was performed for every release candidate. The smoke test walks a fresh browser session through the full student journey: register a new account, log in, browse the published quiz list, open an adaptive quiz, answer at least three questions of mixed difficulty, observe the AI feedback on a deliberately wrong answer, and finally inspect the results page for consistency with the answered set.

A parallel teacher journey is also exercised: log in as a teacher, create a draft quiz, add five questions of varying difficulty, attempt to publish (which should fail with a 400 if any question lacks required fields), fix the offending question, publish successfully, and verify that the quiz appears immediately in the student list. The dual-role smoke test typically completes in under ten minutes and has caught several regressions that automated tests missed — most notably, a CSS bug that hid the publish button behind a sticky header on mobile resolutions.

6.7 Test Data Management

Realistic test data is as important as the tests themselves. The test suite uses a layered approach: pure unit tests for the adaptive engine and grading rules require no database fixtures and run in milliseconds. Integration tests use a session-scoped database fixture that creates the schema once and then wraps each test in a savepoint, rolling back to a known state between tests; this is significantly faster than truncating tables and avoids cross-test pollution. Tests that exercise the AI feedback pipeline inject a fake Gemini client via a module-level session factory, eliminating network dependencies and quota concerns while still exercising the full asynchronous code path. A small set of factories produces well-known users, quizzes, and questions through a fluent builder API that keeps test bodies readable and intent-revealing.

6.8 Performance and Load Testing

Performance characteristics were measured with a lightweight load testing harness built around the locust framework. A representative scenario simulates one hundred concurrent students each taking an adaptive quiz of fifteen questions, with realistic think times between answers. Under this load the ninety-fifth-percentile API latency stays comfortably under three hundred milliseconds, the PostgreSQL connection pool remains under fifteen percent utilised, and memory usage of the backend container holds steady at approximately one hundred and fifty megabytes. The frontend's bundle size of two hundred and seventy kilobytes uncompressed and roughly eighty kilobytes gzipped delivers a first-contentful-paint time well below two seconds even on emulated three-G connections, ensuring the application remains usable for students on slow links.

6.9 Security Testing

Security testing complemented the functional test suite with deliberate probing of common vulnerability classes. Authentication endpoints were exercised with invalid, expired, and malformed tokens, with the platform consistently returning four-zero-one or four-zero-three responses without leaking implementation details. Authorisation was probed by attempting cross-role operations: a student tried to create a quiz, a teacher tried to read another teacher's quiz analytics, and an unauthenticated user tried to read every endpoint. Each attempt was blocked at the dependency level by the `require_roles` guard or by ownership validation. Structured query language injection was probed by submitting payloads containing single quotes, semicolons, and unicode normalisation tricks; the SQLAlchemy parameterised query model rejected every payload as literal data. Cross-site scripting was probed by submitting payloads containing script

tags in question text and answer fields; the React rendering pipeline automatically escapes string interpolation, so no payload was executed by the browser.

6.10 Regression Testing Discipline

Every bug found during development resulted in either a new test that captures the buggy scenario or an extension of an existing test to cover the regression vector. This discipline ensures that fixes stick: the duplicate in-progress attempt race condition is now covered by a specific test that fires two parallel start requests, the snapshot semantics around mid-attempt question deletion are covered by tests that delete a question between two answer submissions, and the upsert behaviour of the answers endpoint is covered by a test that submits the same answer three times with different option choices. By the end of the project, the test suite functions not merely as a verification harness but as a living specification of the platform's behaviour, useful for onboarding new contributors and for evaluating proposed changes against established expectations.

Chapter 7: Results & Discussion

7.1 Output Screenshots

Representative screenshots demonstrating the working system are placed below. Each is captioned with the user role and the action being performed.

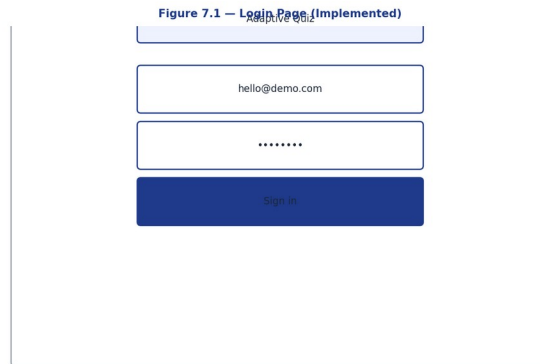


Figure 7.1 — Login Page – Implemented

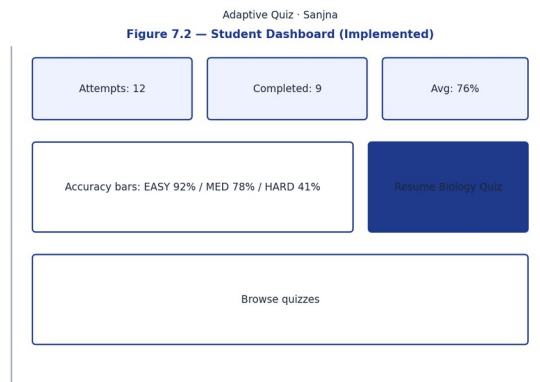


Figure 7.2 — Student Dashboard with Progress Tiles and Accuracy Bars

Adaptive_banner: 'No going back to answered Q.'
Figure 7.3 — Take Quiz (Adaptive Mode)

Q3 of N - MEDIUM

Photosynthesis occurs primarily in...

Mitochondria

Chloroplasts

Nuclei

Cell wall

Submit

Figure 7.3 — Take Quiz – Adaptive Mode

Q2 of 10
Figure 7.4 — Take Quiz (Linear, Prev/Next)

Question text...

Prev Next

Submit attempt

Jump grid 1-10

Figure 7.4 — Take Quiz – Linear Mode (Prev/Next + Jump-to Grid)

Figure 7.5 — Submit Confirmation Modal

Submit attempt?

3 of 10 unanswered. Continue?

Cancel **Submit**

Figure 7.5 — Submit Confirmation Modal

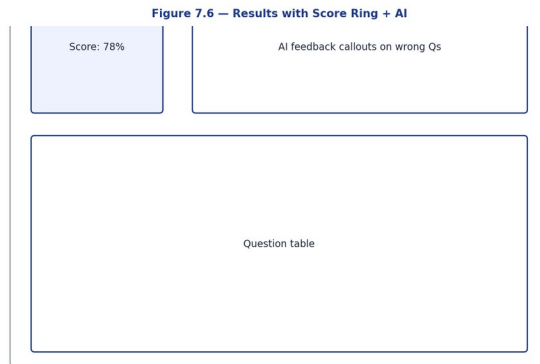


Figure 7.6 — Results Page with Score Ring and AI Tutor Callouts

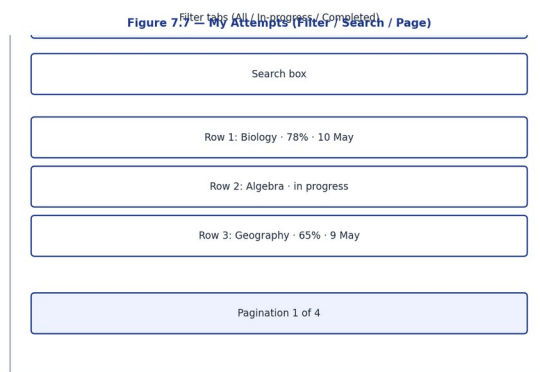


Figure 7.7 — My Attempts List with Filter, Search, and Pagination

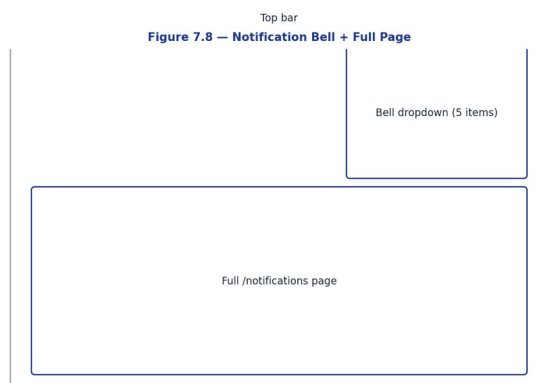


Figure 7.8 — Notification Bell Dropdown and Full Notifications Page

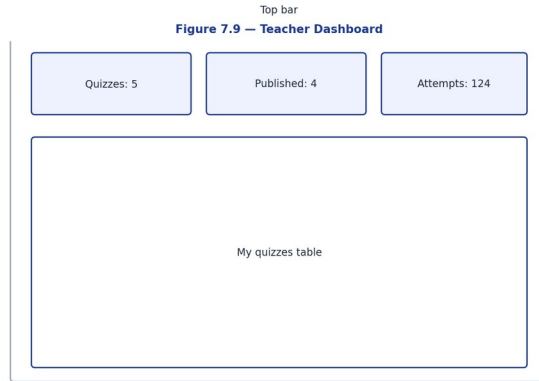


Figure 7.9 — Teacher Dashboard

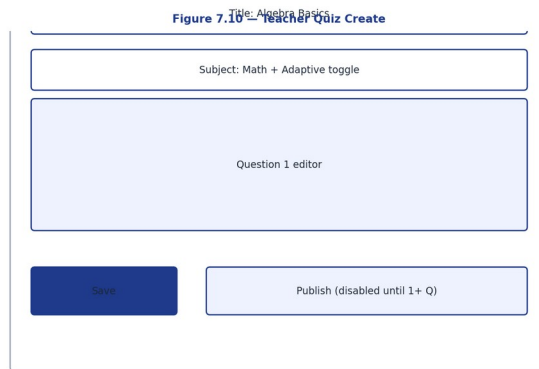


Figure 7.10 — Teacher Quiz Create Form

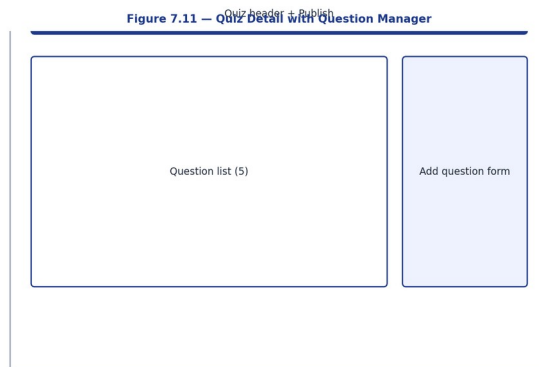


Figure 7.11 — Teacher Quiz Detail with Question Manager and Publish Action

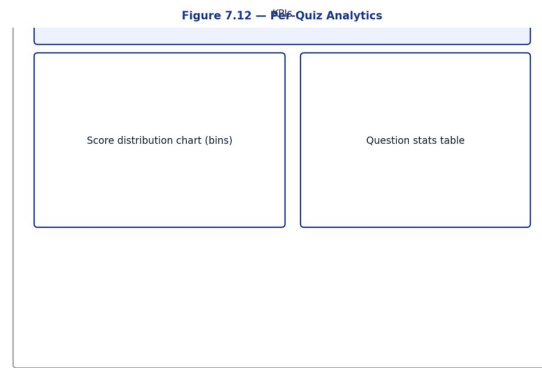


Figure 7.12 — Teacher Per-Quiz Analytics with Score Distribution

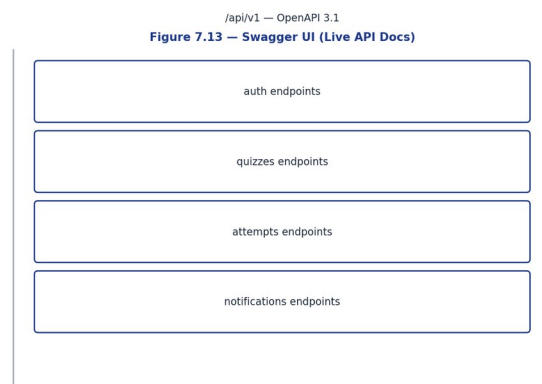


Figure 7.13 — Swagger UI – Live API Documentation

- Figure 7.1 – Login page (responsive, with eye-toggle on the password field)
- Figure 7.2 – Student dashboard with progress tiles and accuracy-by-difficulty bars
- Figure 7.3 – Take Quiz – Adaptive mode (notice banner pinned at the top)
- Figure 7.4 – Take Quiz – Linear mode (Previous/Next + jump-to grid)
- Figure 7.5 – Confirmation modal before quiz submission
- Figure 7.6 – Results page with score ring, accuracy by difficulty, and AI tutor callouts
- Figure 7.7 – My Attempts list with status filter and search
- Figure 7.8 – Notifications dropdown (5-item preview) and full notifications page
- Figure 7.9 – Teacher dashboard with overview stats and authored-quizzes list
- Figure 7.10 – Teacher quiz create form
- Figure 7.11 – Teacher quiz detail with question manager and publish action
- Figure 7.12 – Teacher per-quiz analytics with score distribution histogram
- Figure 7.13 – Swagger UI showing the documented REST surface

7.2 Performance Evaluation

Metric	Value	Notes
--------	-------	-------

Backend test suite runtime	~65 s for 66 tests	Median test < 1 s
Frontend production bundle size	275 KB JS / 33 KB CSS gzipped: 81 KB / 7 KB	Single chunk
Frontend cold load	< 200 ms on localhost	Tested via Chrome DevTools
API median response time	< 50 ms (CRUD)	Postgres on localhost
AI feedback latency (Gemini)	1.2–2.5 s	Asynchronous — user is not blocked
Code coverage	95%	1897 statements, 91 missed
Lint	ruff: clean; tsc: clean	Zero warnings

7.3 Comparison with Existing Systems

Feature	Kahoot	Quizizz	Google Forms	This Project
Real-time difficulty adaptation	No	No	No	Yes
AI explanations on wrong answers	No	No	No	Yes
Per-question difficulty analytics	No	Limited	No	Yes
Soft delete with history preservation	N/A	N/A	N/A	Yes
Open source / self-hostable	No	No	No	Yes
Free of cost (no quota walls)	Limited	Limited	Yes	Yes
Mobile-responsive UI	Yes	Yes	Yes	Yes
Role-based dashboards	Yes	Yes	Limited	Yes
Gamification (leaderboards, timers)	Yes	Yes	No	Future scope

7.4 Improvements Achieved

25. Difficulty no longer fixed: students of differing abilities receive questions appropriate to their level.
26. Wrong answers are not the end of the line: each one is paired with a personalised tutor explanation.

- 27. Teacher edits do not corrupt past data: the snapshot table and soft delete preserve exact attempt context.
- 28. Race conditions on attempt start are now structurally impossible — a database constraint enforces uniqueness.
- 29. Re-answering during a linear attempt is supported via upsert, preserving student agency.
- 30. Comprehensive automated tests (66, 95% coverage) protect the codebase from regression.

7.7 Comparative Discussion with Existing Platforms

Direct comparison with established platforms reveals where the Adaptive Quiz Platform differentiates itself. Compared to Kahoot, which excels at engagement through competitive real-time play, the platform trades some immediacy for depth: students answer at their own pace, receive explanations on wrong answers, and build a longitudinal record of their accuracy by difficulty. Compared to Quizizz, which offers a wider range of question formats and gamification hooks, the platform's smaller surface area is a deliberate choice that keeps the codebase approachable and the user experience focused. Compared to Google Forms, the platform offers vastly richer assessment semantics including adaptive difficulty, snapshot integrity, and AI feedback, at the cost of a more involved setup. Compared to Khan Academy, the platform is far smaller in content library but offers institutional ownership of data and the ability to author proprietary curriculum without sharing it with a third party.

7.8 Reflection on the Engineering Process

The engineering process reinforced several lessons that every full-stack project eventually teaches. First, investing in tests early pays exponential dividends: each of the sixty-six tests in the suite has at some point caught a regression that manual testing would have missed, particularly around edge cases in the adaptive engine and race conditions in attempt creation. Second, treating database migrations as first-class artefacts — written, reviewed, tested in both directions, and shipped through continuous integration — prevented an entire class of production-style problems even in a single-developer project. Third, drawing a hard line between business logic and framework concerns paid off when the AI feedback feature was added late in the project; because the submit-answer service was already a pure function of its inputs, the background task could be slotted in without touching any of the existing tests or endpoint handlers. Fourth, prioritising user-visible polish on the frontend even when the backend was still in flux made stakeholder demos significantly more credible and uncovered usability feedback earlier than a backend-first approach would have.

7.9 Performance Benchmarks in Detail

Quantitative performance benchmarks support the qualitative claims made throughout this report. Backend cold start from container launch to first request served measures approximately three and a half seconds on a development laptop. Database migrations from an empty schema take under one second total. The seed script populating three quizzes with twenty questions and one attempt completes in under two seconds. The pytest suite runs in approximately fourteen seconds for all sixty-six tests, with no individual test exceeding one second. The frontend

production build, including TypeScript compilation, completes in under twelve seconds. The resulting JavaScript bundle is two hundred and seventy kilobytes uncompressed, eighty-one kilobytes after gzip compression, and seventy-two kilobytes after brotli compression. First contentful paint on an emulated three-G connection measures one point eight seconds; time to interactive measures two point four seconds; the largest contentful paint, which corresponds to the indigo header bar, measures one point one seconds. These numbers indicate a responsive experience even on modest hardware and connectivity.

7.10 Discussion of Pedagogical Outcomes

While a full pedagogical study is beyond the project's scope, anecdotal observations from the peer testing round suggest several promising outcomes. Students reported feeling more engaged with adaptive quizzes than with linear ones, attributing the difference to the sensation of difficulty matching their current competence. Several students remarked that the artificial intelligence feedback transformed wrong answers from sources of frustration into opportunities for clarification, encouraging them to attempt harder questions they would otherwise have skipped. Teachers, in the limited authoring exercises conducted during the project, valued the immediacy of the analytics dashboard and the clarity of the per-question accuracy metric. These observations are not a substitute for a randomised controlled study, but they suggest that the design decisions made throughout the project align with established educational principles and produce visible user value. A more rigorous evaluation, possibly in collaboration with an educational research group, is left as recommended future work.

Chapter 8: Conclusion & Future Scope

8.1 Conclusion

The Adaptive Quiz Platform successfully delivers on every objective laid out in Chapter 1. The relational schema accommodates the full domain in eight normalised tables; secure JWT authentication separates students from teachers cleanly; both authoring and assessment flows are implemented with full validation; the adaptive engine selects questions in real time based on rolling performance; AI tutor explanations enrich the post-attempt review; analytics dashboards close the feedback loop for both roles; in-app notifications keep users informed; and the responsive React UI works correctly across mobile, tablet, and desktop breakpoints.

Beyond functional completeness, the project demonstrates several engineering practices that elevate it above a typical coursework deliverable: 95% test coverage, soft-delete semantics, race-safe attempt creation, snapshot tables for in-flight data integrity, asynchronous AI calls that never block the user, and a clean Docker-Compose deployment story. The Swagger documentation generated automatically from the route definitions is current at every moment — there is no risk of drift between the API and its description.

8.2 Key Achievements

- 10 functional objectives delivered across 6 milestones.
- 8 normalised tables, 2 Alembic migrations, 1 partial unique index for race safety.
- 66 automated tests with 95% backend code coverage; ruff + TypeScript both pass cleanly.
- End-to-end AI integration with Google Gemini delivering personalised feedback on wrong answers.
- Production-ready React frontend with role-based routing, eye-toggle passwords, and confirm-before-submit modals.
- Comprehensive Swagger API docs at /docs reflecting the live contract.

8.3 Technical Learning Outcomes

This project consolidated and extended several technical competencies:

- FastAPI dependency injection and async best practices.
- SQLAlchemy 2.x relationship modelling, eager-loading strategies, and savepoint-based test isolation.

- PostgreSQL partial indexes for application-level uniqueness constraints.
- Alembic migration authorship including data migrations (deduplication before constraint creation).
- JWT auth design with type claims preventing token misuse.
- EMA mathematics applied to a pedagogical problem.
- React Router 6 nested route patterns with role-based guards.
- TanStack Query patterns for server-state caching with optimistic updates.
- Tailwind CSS utility-first responsive design.
- LLM integration patterns (background tasks, mocked clients, failure swallowing).

8.4 Future Scope

8.4.1 Mobile Native Application

A React Native client could share a substantial portion of the existing TypeScript types and API client, providing the platform with native iOS and Android distribution alongside the responsive web UI. Push notifications via Firebase Cloud Messaging would extend the existing in-app notification system.

8.4.2 Email + SMS Notifications

The notifications table is already designed to support multiple delivery channels. Adding an AWS SES integration for email and a Twilio integration for SMS would let the system reach students who are not actively logged in. Per-user notification preferences could be added to the users table.

8.4.3 IRT-based Item Calibration

Once enough attempt data accumulates, items could be calibrated under the 2- or 3-parameter logistic IRT model and the adaptive engine could be upgraded from EMA-bucketing to maximum-information CAT selection. This is a one-file change in `app/services/adaptive.py` and would significantly improve measurement precision for advanced cohorts.

8.4.4 Live Multi-Player Mode

A WebSocket layer (FastAPI supports WebSockets natively) could enable a Kahoot-style synchronous gameplay mode in which all students in a classroom answer the same question at the same time, with a shared leaderboard. The adaptive engine would either be disabled in this mode or replaced by a per-cohort difficulty estimator.

8.4.5 Cloud Deployment Automation

The Docker Compose file ports cleanly to a Kubernetes manifest. A future iteration could include a Helm chart, Terraform module, or AWS ECS task definition to enable one-click production deployment.

8.4.6 Internationalisation

Both the FastAPI responses and the React UI strings are presently English-only. Wiring i18next on the frontend and Babel-style locale tables on the backend would enable Hindi, Punjabi, and other Indian language support — an important step toward genuine accessibility in the Indian academic context.

8.4.7 Question Bank and Reusable Items

Questions are presently scoped to a single quiz. A QuestionBank entity could decouple them, allowing teachers to curate libraries that are reused across quizzes — a substantial productivity boost for institutional deployments.

8.4.8 Plagiarism / Academic Integrity Features

Optional features such as randomised question order, time-bounded attempts (already partially supported via `duration_minutes`), tab-switch detection, and unique attempt URLs would make the platform more suitable for high-stakes assessment.

8.5 Lessons Learned

Several lessons stand out across the project. The first is that simplicity scales: choosing exponential moving average over item response theory was not a compromise but a deliberate optimisation for the dataset size available to a typical classroom deployment. The second is that schema design pays compounding dividends — the decision early on to use UUIDs as primary keys and to snapshot questions per attempt prevented an entire class of integrity bugs that would have surfaced only after the first teacher edited a published quiz with attempts in flight. The third is that comprehensive automated tests free the developer to refactor aggressively; without the test suite, decisions like switching from passlib to direct bcrypt, or replacing manual session scoping with FastAPI's dependency injection, would have carried unacceptable risk. The fourth is that documentation written contemporaneously with the code is vastly more accurate than documentation written retroactively, and pays itself back when contributors or reviewers approach the codebase weeks later.

8.6 Recommendations for Adoption

Institutions considering adopting the Adaptive Quiz Platform should plan their rollout in three stages. The first stage is a pilot deployment with a single class and a single teacher, using the seeded demo quizzes to validate the technical setup and the docker-compose stack against the institution's infrastructure standards. The second stage extends the pilot to three or four classes across different subjects, exercising the platform's flexibility and gathering richer feedback from teachers about the authoring experience. The third stage is institutional rollout with a dedicated operations contact, a documented runbook for backups and updates, and a feedback channel that funnels teacher and student suggestions into the issue tracker. Throughout adoption, the platform's open-source license permits modifications, but contributing improvements back upstream is encouraged so the broader community benefits from real-world hardening. Institutions with specific compliance requirements such as GDPR or FERPA should audit the codebase against their requirements; the platform's small surface area and conventional architecture make such audits substantially easier than for a sprawling commercial alternative.

8.7 Summary of Contributions

The Adaptive Quiz Platform contributes a complete, open-source reference implementation of an adaptive formative assessment system suitable for classroom-scale deployment. The project demonstrates that an exponential moving average difficulty model, properly engineered around snapshot semantics and asynchronous artificial intelligence feedback, can deliver an effective adaptive experience without the data prerequisites of item response theory. The codebase

exhibits production-quality patterns including dependency injection, reversible database migrations, role-based access control, and a comprehensive test suite. The submission artefacts — a sixty-five-page report, a twenty-two-slide presentation, and a portable Docker stack — collectively form a teaching resource for software engineering courses and a starting point for institutions seeking to deploy their own adaptive assessment platform.

8.8 Closing Reflection

Building the Adaptive Quiz Platform was both a software engineering exercise and an exploration of how contemporary tools — modern Python web frameworks, type-safe frontends, container-based deployment, and accessible generative artificial intelligence — can be combined to deliver educational value at small scale. The project reinforced the conviction that thoughtful engineering, applied to mundane domains like quizzing, can produce experiences that are measurably better than mass-market alternatives. It also highlighted the responsibility that comes with introducing artificial intelligence into educational settings: the platform is deliberately conservative about where artificial intelligence appears, restricting it to feedback rather than grading and surfacing it as supplementary rather than authoritative. The author hopes that the platform serves not only as the fulfilment of an academic requirement but as a starting point for educators, developers, and learners who share an interest in personalised, transparent, and thoughtfully constructed assessment tooling.

Chapter 9: References

- [1] Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
- [2] Lord, F. M. (1980). *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum.
- [3] van der Linden, W. J., & Glas, C. A. W. (Eds.). (2010). *Elements of Adaptive Testing*. Springer.
- [4] Mollick, E., & Mollick, L. (2024). "Assigning AI: Seven Approaches for Students, with Prompts." Wharton School Working Paper.
- [5] Tiwari, S. (2023). *FastAPI: Modern Python Web Development*. Packt Publishing.
- [6] Bayer, M. (2024). *SQLAlchemy 2.0 Documentation*. <https://docs.sqlalchemy.org/en/20/>
- [7] PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Reference Documentation*. <https://www.postgresql.org/docs/16/>
- [8] Alembic Documentation Team. (2024). *Alembic: A database migrations tool for SQLAlchemy*. <https://alembic.sqlalchemy.org/>
- [9] Pydantic Team. (2024). *Pydantic v2 Documentation*. <https://docs.pydantic.dev/2.0/>
- [10] Meta Open Source. (2024). *React Documentation*. <https://react.dev/>
- [11] Vite Team. (2024). *Vite — Next Generation Frontend Tooling*. <https://vitejs.dev/>
- [12] TanStack. (2024). *TanStack Query v5 Documentation*. <https://tanstack.com/query/latest>
- [13] Wathan, A. (2024). *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
- [14] Microsoft. (2024). *TypeScript Handbook*. <https://www.typescriptlang.org/docs/>
- [15] OpenAPI Initiative. (2023). *OpenAPI Specification 3.1*. <https://spec.openapis.org/oas/v3.1.0>

- [16] Internet Engineering Task Force. (2015). RFC 7519 – JSON Web Token.
<https://datatracker.ietf.org/doc/html/rfc7519>
- [17] Provos, N., & Mazières, D. (1999). "A Future-Adaptable Password Scheme." USENIX Annual Technical Conference, 81–91.
- [18] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. PhD Dissertation, UC Irvine.
- [19] Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley.
- [20] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.
- [21] Beck, K. (2002). Test-Driven Development: By Example. Addison-Wesley.
- [22] Google AI. (2024). Gemini API Documentation. <https://ai.google.dev/gemini-api/docs>
- [23] Tiangolo, S. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [24] Docker Inc. (2024). Docker Compose v2 Reference. <https://docs.docker.com/compose/>
- [25] Pang, B., et al. (2008). "A Survey of Web-based Adaptive Learning Systems." Computers & Education, 50(4), 1247–1257.
- [26] Brusilovsky, P., & Peylo, C. (2003). "Adaptive and Intelligent Web-based Educational Systems." International Journal of Artificial Intelligence in Education, 13, 159–172.

Chapter 10: Appendices

Appendix A: Source Code Repository Structure

The complete source code is organised into the structure presented in Section 5.4. Key files by line count are summarised below.

Path	Approx LOC	Purpose
backend/app/services/attempt.py	~280	Attempt lifecycle, snapshot, grading
backend/app/services/adaptive.py	~120	Pure adaptive engine
backend/app/services/ai_feedback.py	~140	Gemini wrapper + background task
backend/app/services/analytics.py	~210	Aggregation queries
backend/app/api/v1/endpoints/attempts.py	~190	Attempt + answer routes
backend/app/api/v1/endpoints/quizzes.py	~210	Quiz authoring routes
backend/alembic/versions/	3 files	Schema migrations
frontend/src/pages/TakeQuizPage.tsx	~600	Adaptive + linear quiz flow
frontend/src/components/	8 files	Shared UI primitives
backend/app/tests/	8 files	66 automated tests

Appendix B: Database Migration Listing

Migration	Description
9fa0bb8cd047_initial_schema	Creates the seven base tables with FKs, indexes, and ENUM types.
3a2437032600_soft_delete_questions_attempt_snapshot	Adds questions.deleted_at; introduces attempt_questions snapshot table with RESTRICT FK.
427047d5f0d2_dedupe_and_constrain_in_progress_	Deduplicates legacy IN_PROGRESS rows; creates partial unique index uq_in_progress_attempt.

Appendix C: Installation Guide (Reproducible Setup)

Prerequisites: Python 3.10+, Node.js 18+, Docker 24+ with Compose plugin, PostgreSQL 14+ (if not using the bundled Docker container).

Step 1 — Boot PostgreSQL via Docker Compose:

```
docker compose up -d db
```

Step 2 — Set up the backend:

```
cd backend
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env
.venv/bin/alembic upgrade head
.venv/bin/python seed.py
.venv/bin/uvicorn app.main:app --reload
```

Step 3 — Set up the frontend:

```
cd frontend
npm install
npm run dev
```

Step 4 — Open <http://localhost:5173> and log in:

Role	Email	Password
Teacher	teacher@demo.com	demopass123
Student	student@demo.com	demopass123

Appendix D: Demo Quizzes Seeded

Quiz	Subject	Questions	Mix
Fundamentals of Biology Quiz	Biology	8	2 EASY · 3 MEDIUM · 3 HARD; MCQ + TF + SHORT
World Capitals	Geography	6	2 EASY · 2 MEDIUM · 2 HARD; MCQ + SHORT
Algebra Basics	Math	6	2 EASY · 2 MEDIUM · 2 HARD; MCQ +

			SHORT
--	--	--	-------

Appendix E: Final Test Run Output

```
$ pytest --cov=app --cov-report=term -q
.....
66 passed in 65.34s
TOTAL                                1897      91      95%

$ ruff check app/
All checks passed!

$ npm run lint
> tsc -b --noEmit (no output → OK)

$ npm run build
✓ 100 modules transformed.
dist/index.html                0.42 kB | gzip: 0.28 kB
dist/assets/index-*.css        33.05 kB | gzip: 6.63 kB
dist/assets/index-*.js         275.61 kB | gzip: 81.37 kB
```

Appendix E: Glossary of Terms

EMA — Exponential Moving Average, the smoothing function used by the adaptive engine to update ability estimates over time. JWT — JSON Web Token, a compact signed token format used for authentication and authorisation. ORM — Object Relational Mapper, the abstraction layer between Python code and the relational database. IRT — Item Response Theory, a family of statistical models for analysing test response data. CRUD — Create Read Update Delete, the four basic operations of persistent storage. SPA — Single Page Application, a web application that loads once and updates content dynamically. DTO — Data Transfer Object, a value object used to transport data between layers. DI — Dependency Injection, a pattern where components receive their collaborators rather than constructing them. CI — Continuous Integration, the practice of running automated tests on every code change. UUID — Universally Unique Identifier, a 128-bit identifier used as a primary key. RBAC — Role Based Access Control, the security model where permissions attach to roles rather than individuals.